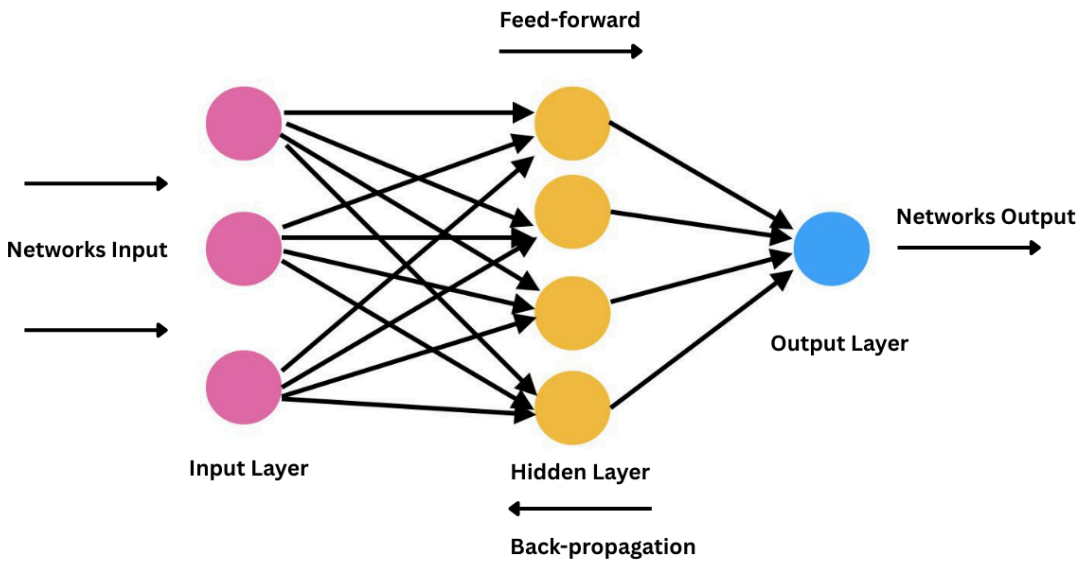
Tugas Besar 1 IF3270 Pembelajaran Mesin

Feedforward Neural Network



Dipersiapkan oleh:

William Andrian Dharma T 13523006

Varel Tiara 13523008

Nathan Jovial Hartono 13523032

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

2025

Daftar Isi

Daftar Isi **2**
1 Deskripsi Persoalan **3**
2 Pembahasan **5**
2.1 Penjelasan Implementasi 5
2.2 Hasil Pengujian 20
3 Kesimpulan dan Saran **48**
3.1 Kesimpulan 48
3.2 Saran 49
4 Pembagian tugas tiap anggota kelompok **51**
5 Referensi **52**
6 Lampiran Form Penggunaan AI **53**

1 Deskripsi Persoalan

Tugas Besar 1 mata kuliah IF3270 Pembelajaran Mesin ini berfokus pada implementasi Feedforward Neural Network (FFNN) from scratch menggunakan bahasa Python. Permasalahan utamanya adalah bagaimana membangun suatu model yang mampu melakukan proses pembelajaran melalui mekanisme forward propagation, backward propagation, serta weight update menggunakan algoritma gradient descent. Implementasi ini bertujuan untuk memahami secara mendalam proses komputasi yang terjadi di dalam model neural network, mulai dari nilai propagation antar layer, perhitungan fungsi aktivasi, hingga proses optimisasi bobot berdasarkan nilai gradien terhadap fungsi loss.

Model FFNN yang dibangun juga harus mampu menerima konfigurasi arsitektur jaringan yang berbeda, seperti jumlah layer, jumlah neuron pada tiap layer (termasuk input layer dan output layer), serta pilihan fungsi aktivasi dari tiap layer. Fungsi aktivasi yang diimplementasikan adalah Linear, ReLU, Sigmoid, Hyperbolic Tangent (tanh), dan Softmax. Selain itu, model ini juga mampu menerima fungsi loss seperti Mean Squared Error (MSE), Binary Cross-Entropy, dan Categorical Cross-Entropy. Implementasi tersebut juga mencakup mekanisme inisialisasi bobot tiap neuron, termasuk bias menggunakan beberapa metode, seperti zero initialization, random initialization dengan distribusi uniform, serta random initialization dengan distribusi normal dengan parameter tertentu seperti batas nilai, mean, variance, dan seed untuk reproducibility. Model yang diinisialisasi juga harus mampu menyimpan nilai bobot serta gradien bobot pada setiap neuron selama proses pelatihan.

Dalam proses pelatihan model, sistem harus mengimplementasikan forward propagation yang mampu menerima input dalam bentuk batch. Selanjutnya, sistem juga harus mengimplementasikan backward propagation untuk menghitung perubahan gradien bobot terhadap fungsi loss menggunakan konsep chain rule pada turunan parsial. Gradien yang diperoleh kemudian digunakan dalam proses pembaruan bobot menggunakan metode gradient descent. Model juga harus menyediakan metode untuk menampilkan distribusi bobot dan distribusi gradien pada layer tertentu, serta mampu untuk save dan load model yang telah dilatih.

Sebagai bagian dari proses pelatihan, implementasi model harus mendukung beberapa parameter pelatihan seperti batch size, learning rate, jumlah epoch, serta verbose mode yang mengatur tingkat informasi yang ditampilkan selama proses pelatihan berlangsung. Proses pelatihan juga harus menghasilkan histori pelatihan yang berisi nilai training loss dan validation loss pada setiap epoch.

Setelah modul FFNN berhasil diimplementasikan, model akan diuji menggunakan dataset Global Student Placement & Salary Dataset. Dataset ini berisi kumpulan data mengenai performa akademik mahasiswa serta outcome penempatan kerja setelah lulus. Beberapa fitur yang terdapat dalam dataset adalah cgpa, backlogs, college_tier, country, university_ranking_band, internship_count, aptitude_score, communication_score, specialization, industry, serta

internship_quality_score. Berdasarkan fitur-fitur tersebut, model diharapkan mampu memprediksi placement_status, yaitu status penempatan kerja mahasiswa setelah lulus.

Selain implementasi model, tugas ini juga mencakup tahap pengujian dan eksperimen untuk menganalisis pengaruh beberapa hyperparameter terhadap performa model. Eksperimen yang dilakukan meliputi analisis pengaruh depth (banyak layer) dan width (banyak neuron per layer), dengan menggunakan tiga variasi kombinasi width dengan depth tetap serta tiga variasi depth dengan width yang sama pada setiap layer. Selain itu, dilakukan pula eksperimen untuk menganalisis pengaruh fungsi aktivasi pada hidden layer dengan menentukan sebuah base arsitektur minimal tiga layer dan memilih satu layer sebagai layer pengujian. Variasi fungsi aktivasi yang telah diimplementasikan, kecuali softmax, diterapkan pada layer tersebut untuk melihat pengaruhnya terhadap performa model. Eksperimen selanjutnya menganalisis pengaruh learning rate dengan menggunakan tiga variasi nilai learning rate yang berbeda, serta pengaruh regularisasi dengan membandingkan model tanpa regularisasi, dengan regularisasi L1, dan dengan regularisasi L2.

Hasil dari setiap eksperimen dianalisis dengan membandingkan hasil prediksi model, grafik training loss dan validation loss pada setiap epoch pelatihan, serta distribusi bobot dan gradien bobot pada beberapa layer jaringan. Sebagai pembanding, performa model FFNN yang diimplementasikan juga dibandingkan dengan model neural network yang tersedia pada library Scikit-Learn, yaitu MLPClassifier, dengan melakukan satu kali pelatihan menggunakan hyperparameter yang sama pada kedua model dan kemudian membandingkan hasil prediksi yang dihasilkan.

2 Pembahasan

2.1 Penjelasan Implementasi

2.1.1 Kelas

1. class Tensor: Wrapper sederhana untuk array NumPy. Digunakan sebagai tipe data standar untuk menyimpan data, bobot, dan bias di seluruh modul.

Atribut	Tipe	Deskripsi
data	np.ndarray	Array Numpy yang menyimpan data tensor

Method	Deskripsi
__init__(self, data)	Mengonversi input ke np.ndarray dan menyimpannya di self.data
dim()	Mengembalikan shape (dimensi) dari tensor, berupa tuple

2. class loss: Kelas abstrak yang mendefinisikan interface untuk semua fungsi loss.

Method	Deskripsi
get_loss(input, target)	Menghitung nilai loss antara prediksi dan target
get_gradient(input, target)	Menghitung gradien loss terhadap prediksi

3. class Layer: Kelas abstrak yang mendefinisikan interface untuk semua layer.

Method	Deskripsi
forward(x: Tensor)	Melakukan forward pass, menerima input dan menghasilkan output
backward(grad, lr)	Melakukan backward pass, menerima gradien dari layer sesudahnya, mengembalikan gradien ke layer sebelumnya
print_weights()	Menampilkan bobot layer (optional)
print_gradients()	Menampilkan gradien layer (opptional)

2.1.2 Fungsi Loss

1. class MSE(loss): Mean Squared Error yang digunakan untuk tugas regresi. Mengukur rata-rata kuadrat selisih antara prediksi dan target.

   Rumus loss:

$$\text{MSE} = \frac{1}{N \cdot D} \sum_{i,j} (\hat{y}_{ij} - y_{ij})^2$$

```
def get_loss(self, input, target):
    return np.mean((input - target) ** 2)
```

   np.mean menghitung rata-rata seluruh elemen (batch × output_dim).

   Rumus gradien:

$$\frac{\partial \text{MSE}}{\partial \hat{y}_{ij}} = \frac{2(\hat{y}_{ij} - y_{ij})}{N \times D}$$

```
def get_gradient(self, input, target):
    return 2 * (input - target) / input.size
```

   Membagi dengan input.size (= N × D) agar konsisten dengan np.mean di get_loss.

2. class BinaryCrossEntropyLoss(loss): Digunakan untuk tugas klasifikasi biner (2 kelas). Target berupa nilai 0 atau 1.

   Rumus loss:

$$\text{BCE} = -\frac{1}{N} \sum_{i} \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

```
def get_loss(self, input, target):
    input = np.clip(input, 1e-9, 1 - 1e-9)  # cegah log(0)
    return -np.mean(target * np.log(input) + (1 - target) * np.log(1 -
input))
```

np.clip memastikan prediksi tidak persis 0 atau 1 sehingga np.log tidak menghasilkan -inf.

Rumus gradien:

$$\frac{\partial \text{BCE}}{\partial \hat{y}_i} = \frac{\hat{y}_i - y_i}{\hat{y}_i(1 - \hat{y}_i) \cdot N}$$

```
def get_gradient(self, input, target):
    input = np.clip(input, 1e-9, 1 - 1e-9)
    return (input - target) / (input * (1 - input) * input.shape[0])
```

3. class CrossEntropyLoss(loss): Categorical Cross-Entropy yang digunakan untuk tugas klasifikasi multi-kelas. Target berupa integer (label encoding), bukan one-hot.
   Rumus loss:

$$\text{CCE} = -\frac{1}{N} \sum_i \log(\hat{y}_{i,c_i})$$

di mana ci adalah kelas target dari sampel ke-i.

```
def get_loss(self, input, target):
    N = input.shape[0]
    y_pred = input[range(N), target]
    return np.mean(-np.log(y_pred + 1e-9))
```

input[range(N), target] menggunakan fancy indexing, untuk setiap sampel, hanya probabilitas kelas yang benar yang diambil.

Rumus gradien:

Ketika dipasangkan dengan Softmax, gradiennya menjadi sangat sederhana:

$$\frac{\partial \text{CCE}}{\partial z_{ij}} = \frac{\hat{y}_{ij} - \mathbf{1}[j = c_i]}{N}$$

```
def get_gradient(self, input, target):
    N = input.shape[0]
    self.grad = input.copy()
    self.grad[range(N), target] -= 1
    self.grad /= N
    return self.grad
```

Gradien ini merupakan hasil kombinasi turunan softmax + cross-entropy. Untuk setiap sampel, semua elemen gradien = probabilitas prediksi, kecuali pada posisi kelas target dikurang 1.

2.1.3 Inisialisasi Bobot

class init: Kumpulan method statis untuk menginisialisasi bobot pada objek Tensor.

Method	Deskripsi
zeros(tensor)	Mengisi semua nilai tensor dengan 0
uniform(tensor, lower_bound, upper_bound, seed)	Random uniform dalam [lower, upper]. Seed untuk reproducibility
normal(tensor, mean, variance, seed)	Random normal dengan mean dan variance tertentu. Std dev = √variance
kaiming_uniform(tensor)	He initialization: bound = √(6/fan_in) yang cocok digunakan untuk ReLU
xavier_uniform(tensor)	Xavier: bound = √(6/(fan_in+fan_out)) yang cocok digunakan untuk sigmoid/tanh

```
def kaiming_uniform(tensor: Tensor):
    fan_in = tensor.data.shape[1] if tensor.data.ndim > 1 else tensor.data.shape[0]
    bound = math.sqrt(6.0 / fan_in)
    tensor.data[:] = np.random.uniform(-bound, bound, tensor.dim())
```

- fan_in = jumlah neuron dari layer sebelumnya (jumlah kolom matriks bobot)
- Bound dihitung berdasarkan fan_in aja yang dirancang agar varians output tetap terjaga saat menggunakan ReLU

```
def xavier_uniform(tensor: Tensor):
    fan_in = tensor.data.shape[1] if tensor.data.ndim > 1 else tensor.data.shape[0]
    fan_out = tensor.data.shape[0] if tensor.data.ndim > 1 else
tensor.data.shape[0]
```

```
bound = math.sqrt(6.0 / (fan_in + fan_out))
tensor.data[:] = np.random.uniform(-bound, bound, tensor.dim())
```

- Memperhitungkan fan_in dan fan_out sehingga distribusi sinyal terjaga baik saat forward maupun backward

2.1.4 Optimizer

1. class Optimizer

Atribut	Tipe	Deskripsi
lr	float	Learning rate

Method	Deskripsi
update(layer, t)	Memperbarui bobot layer pada timestep t.

2. class SGD(Optimizer): Stochastic Gradient Descent yang merupakan optimizer paling dasar dengan memperbarui bobot searah negatif gradien.

   Rumus update:

$$W \leftarrow W - \eta \cdot \nabla W$$

$$b \leftarrow b - \eta \cdot \nabla b$$

```
def update(self, layer, t):
    layer.weights.data -= self.lr * layer.dw     # update bobot
    if layer.bias is not None and layer.db is not None:
        layer.bias.data -= self.lr * layer.db    # update bias
```

3. class Adam(Optimizer): Adaptive Moment Estimation yang merupakan optimizer adaptif yang menggabungkan momentum (rerata bergerak gradien) dan RMSProp (rerata bergerak kuadrat gradien).

Atribut	Tipe	Deskripsi
beta1	float	Koefisien peluruhan untuk momen pertama (default: 0.9)

beta2	float	Koefisien peluruhan untuk momen kedua (default: 0.999)
epsilon	float	Konstanta stabilitas numerik (default: 1e-8)
m_w, m_b	dict	Estimasi momen pertama (mean gradien) untuk bobot dan bias
v_w, v_b	dict	Estimasi momen kedua (varians gradien) untuk bobot dan bias

Rumus update untuk bobot, bias tetap sama:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \text{(bias correction)}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad \text{(bias correction)}$$

$$W \leftarrow W - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

```
# Update momen pertama dan kedua
self.m_w[layer_id] = self.beta1 * self.m_w[layer_id] + (1 - self.beta1) *
layer.dw
self.v_w[layer_id] = self.beta2 * self.v_w[layer_id] + (1 - self.beta2) *
(layer.dw**2)

# Bias correction
m_w_hat = self.m_w[layer_id] / (1 - self.beta1**t)
v_w_hat = self.v_w[layer_id] / (1 - self.beta2**t)

# Update bobot
layer.weights.data -= self.lr * m_w_hat / (np.sqrt(v_w_hat) + self.epsilon)
```

Adam lebih baik dari SGD biasa karena setiap parameternya memiliki learning rate yang efektif masing-masing, menghaluskan update sehingga lebih stabil, dan memperbaiki estimasi di awal training saat momen masih mendekati 0.

2.1.5 Layer

class Linear(Layer): Layer utama dalam FFNN yang akan melakukan transformasi linear: y = xM^T + b.

Atribut	Tipe	Deskripsi
in_features	int	Jumlah neuron input
out_features	int	Jumlah neuron output
weights	Tensor	Matriks bobot, shape (out_features, in_features)
bias	Tensor | None	Vektor bias, shape (out_features,). None jika bias=False
X	Tensor	Input yang disimpan saat forward pass (dibutuhkan untuk backward)
grad / dw	np.ndarray	Gradien terhadap bobot (disimpan setelah backward)
db	np.ndarray	Gradien terhadap bias (disimpan setelah backward)

Constructor, __init__(in_features, out_features, bias=True):

```
def __init__(self, in_features, out_features, bias=True):
    self.in_features = in_features
    self.out_features = out_features
    self.weights = Tensor(np.empty((out_features, in_features)))
    self.bias = Tensor(np.empty(out_features)) if bias else None
    init.xavier_uniform(self.weights)   # Xavier init sebagai default
    if self.bias is not None:
        init.zeros(self.bias)          # Bias diinisialisasi 0
```

Forward, forward(x):

```
def forward(self, x):
    self.X = x                          # simpan input untuk backward
    y = x.data @ self.weights.data.T    # perkalian matriks: (N, in) × (in, out) =
(N, out)
    if self.bias is not None:
        y = y + self.bias.data          # broadcast: (N, out) + (out,)
    return Tensor(y)
```

Operasi: y = X × W^T + b

- x.data berukuran (N, in_features), N adalah batch size
- self.weights.data.T berukuran (in_features, out_features)
- Hasil y berukuran (N, out_features)

Backward, backward(grad, lr, reg_type, reg_lambda, optimizer, t):

```
def backward(self, grad, lr, reg_type=None, reg_lambda=0.0, optimizer=None,
t=1):
    dw = grad.T @ self.X.data    # gradien terhadap bobot: (out, N) × (N, in) =
(out, in)
    db = np.sum(grad, axis=0)    # gradien terhadap bias: sum over batch →
(out,)
    dx = grad @ self.weights.data # gradien untuk layer sebelumnya: (N, out) ×
(out, in) = (N, in)

    # Regularisasi ditambahkan ke gradien bobot
    if reg_type == "l1":
        dw += reg_lambda * np.sign(self.weights.data)
    elif reg_type == "l2":
        dw += 2 * reg_lambda * self.weights.data

    self.grad = dw  # simpan gradien
    self.dw = dw
    self.db = db

    # Update bobot
    if optimizer is not None:
        optimizer.update(self, t)            # gunakan optimizer (Adam/SGD)
    else:
        self.weights.data -= lr * dw         # gradient descent manual
        if self.bias is not None and db is not None:
            self.bias.data -= lr * db

    return dx  # kembalikan gradien ke layer sebelumnya
```

2.1.6 Fungsi Aktivasi

1. class Relu(Layer):

   Rumus

$$\mathrm{ReLU}(x) = \max(0, x) \qquad \mathrm{ReLU}'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

```
def forward(self, x):
    self.mask = (x.data > 0).astype(float)  # mask bernilai 1 jika x>0, 0 jika x≤0
    return Tensor(x.data * self.mask)       # elemen negatif menjadi 0

def backward(self, grad, lr):
    return grad * self.mask  # gradien diteruskan hanya jika mask=1 (x>0)
```

self.mask disimpan saat forward dan dipakai ulang saat backward.

2. class LeakyRelu(Layer): Variasi ReLU yang tidak "mematikan" neuron dengan input negatif.

   Rumus:

$$\text{LeakyReLU}(x) = \begin{cases} x & x > 0 \\ \alpha x & x \leq 0 \end{cases}$$

Atribut	Default	Deskripsi
alpha	0.01	Kemiringan (slope) untuk input negatif

```
def forward(self, x):
    self.mask = np.where(x.data > 0, 1.0, self.alpha)  # 1.0 atau alpha
    return Tensor(x.data * self.mask)

def backward(self, grad, lr):
    return grad * self.mask
```

3. class ELU(Layer): Exponential Linear Unit yang mirip dengan LeakyReLU tetapi menggunakan fungsi eksponensial untuk bagian negatif, menghasilkan kurva yang lebih halus.

   Rumus:

$$\mathrm{ELU}(x) = \begin{cases} x & x > 0 \\ \alpha(e^x - 1) & x \leq 0 \end{cases} \qquad \mathrm{ELU}'(x) = \begin{cases} 1 & x > 0 \\ \alpha e^x & x \leq 0 \end{cases}$$

```
def forward(self, x):
    self.input = x.data
    out = np.where(x.data > 0, x.data, self.alpha * (np.exp(x.data) - 1))
    return Tensor(out)

def backward(self, grad, lr):
    dx = np.where(self.input > 0, 1.0, self.alpha * np.exp(self.input))
    return grad * dx
```

4. class Sigmoid(Layer):

   Rumus:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \qquad \sigma'(x) = \sigma(x)(1 - \sigma(x))$$

```
def forward(self, x):
    self.output = 1 / (1 + np.exp(-x.data))  # hitung sigmoid
    return Tensor(self.output)

def backward(self, grad, lr):
    sigmoid_deriv = self.output * (1 - self.output)  # turunan sigmoid
    return_grad = grad * sigmoid_deriv
    return return_grad                    # chain rule
```

   Turunan sigmoid hanya membutuhkan output (bukan input), sehingga cukup menyimpan self.output saat forward.

5. class Tanh(Layer):

   Rumus:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \qquad \tanh'(x) = 1 - \tanh^2(x)$$

```
def forward(self, x):
    self.output = np.tanh(x.data)
    return Tensor(self.output)

def backward(self, grad, lr):
    tanh_deriv = 1.0 - (self.output ** 2)  # turunan dari tanh
    return_grad = grad * tanh_deriv
    return return_grad
```

6. class Softmax(Layer): Mengonversi vektor logit menjadi distribusi probabilitas (semua elemen ≥ 0, total = 1).

   Rumus:

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

```
def forward(self, x):
    # Numerical stability: kurangi max agar e^(x-max) tidak overflow
    shifted_logit = x.data - np.max(x.data, axis=1, keepdims=True)
    exps = np.exp(shifted_logit)
    self.sum_of_exp = np.sum(exps, axis=1, keepdims=True)
    self.output = exps / self.sum_of_exp   # probabilitas output
    return Tensor(self.output)
```

Shift diperlukan karena jika input seperti [1000, 1001, 1002] dan tanpa shift e^1000 akan overflow dan jika dengan shift akan aman. Hasil softmax tersebut jugag tetap sama karena

$$\frac{e^{x_i - c}}{\sum_j e^{x_j - c}} = \frac{e^{x_i}/e^c}{\sum_j e^{x_j}/e^c} = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

```
def backward(self, grad, lr):
    sum_of_grads_dot_output = np.sum(grad * self.output, axis=1,
keepdims=True)
    return self.output * (grad - sum_of_grads_dot_output)
```

Jacobian Softmax:

: $\frac{\partial S_i}{\partial z_j} = S_i(\delta_{ij} - S_j)$

Saat mengalikan dengan upstream gradient g:

$$\frac{\partial L}{\partial z_i} = \sum_j g_j \cdot S_j(\delta_{ij} - S_i) = S_i \left(g_i - \sum_j g_j S_j \right)$$

Yang diimplementasikan sebagai self.output * (grad - sum_of_grads_dot_output).

7. class LinearActivation(Layer): Fungsi aktivasi identitas yang output = input dan berguna sebagai baseline atau untuk output layer pada regresi.

```
def forward(self, x):
    self.X = x
    return Tensor(x.data.copy())  # copy agar tidak mengubah input
asli

def backward(self, grad, lr):
    return grad  # turunan = 1, jadi gradien diteruskan apa adanya
```

2.1.7 Normalisasi (RMSNorm)

class RMSNorm(Layer): Root Mean Square Normalization yang melakukan normalisasi berdasarkan root mean square dari input. Lebih sederhana dari LayerNorm karena tidak memerlukan penghitungan mean.

Atribut	Tipe	Deskripsi
in_features	int	Dimensi fitur input
eps	float	Konstanta stabilitas numerik (default: 1e-8)
weights	Tensor	Parameter skala (gain), diinisialisasi 1, shape (in_features)
X	np.ndarray	Input asli (disimpan saat forward)
rms	np.ndarray	Nilai RMS dari input
x_norm	np.ndarray	Input yang sudah dinormalisasi

Rumus:

$$\mathrm{RMS}(x) = \sqrt{\frac{1}{D}\sum_{i=1}^{D} x_i^2 + \epsilon}$$

$$\hat{x} = \frac{x}{\mathrm{RMS}(x)}$$

$$y = \gamma \cdot \hat{x}$$

Constructor, __init__(in_features, out_features, bias=True):

```
def forward(self, x):
    self.X = x.data
    self.rms = np.sqrt(np.mean(self.X**2, axis=-1, keepdims=True) + self.eps)
    self.x_norm = self.X / self.rms
    return Tensor(self.weights.data * self.x_norm)
```

```
def backward(self, grad, lr, reg_type=None, reg_lambda=0.0, optimizer=None,
t=1):
    dx_norm = grad * self.weights.data
    self.dw = np.sum(grad * self.x_norm, axis=0)  # gradien terhadap gamma
    self.db = None

    D = self.X.shape[-1]
    dx = (dx_norm / self.rms) - (self.X / (D * self.rms**3)) * np.sum(
        dx_norm * self.X, axis=-1, keepdims=True
    )

    if optimizer is not None:
        optimizer.update(self, t)
    else:
        self.weights.data -= lr * self.dw
    return dx
```

2.1.8 Model

class Model: Root kelas utama yang merangkai semua layer menjadi satu jaringan neural lengkap.

Atribut	Tipe	Deskripsi

layers	list[Layer]	Daftar layer secara berurutan (termasuk aktivasi)
loss	loss	Fungsi loss yang digunakan
optimizer	Optimizer | None	Optimizer yang digunakan (diset saat fit())

Method	Deskripsi
forward(X)	Menjalankan forward propagation melalui semua layer
backward(grad, lr, reg_type, reg_lambda, optimizer, t)	Menjalankan backward propagation melalui semua layer (terbalik)
fit(X,y,epochs,batch_size,lr,penalty,lambda_,optimizer,verbose,seed,validation_data,)	Training loop lengkap
predict(X)	Prediksi kelas (argmax dari output)
show_weights(layer_idx)	Print bobot dari layer tertentu
show_gradients(layer_idx)	Print gradien dari layer tertentu
plot_weights(layer_idx)	Plot histogram distribusi bobot
plot_gradients(layer_idx)	Plot histogram distribusi gradien
save_model(filename)	Simpan model ke file (joblib)
load_model(filename)	Muat model dari file (joblib)

2.1.9 Penjelasan Forward Propagation

Forward propagation adalah proses menghitung output model dari input. Data mengalir dari input layer → hidden layers → output layer secara berurutan.

```
def forward(self, X: Tensor) -> Tensor:
    for layer in self.layers:
        X = layer.forward(X)
    return X
```

Misalkan model:

```
model = Model(
    layers=[Linear(4, 4), Relu(), Linear(4, 3), Softmax()],
    loss=CrossEntropyLoss()
)
```

Dengan input X berukuran batch 32 sampel, 4 fitur:

Input X: (32,4) -> Linear(4,4), $z_1 = X @ W_1^T + b_1$, Shape: $(32,4) \times (4,4)^T + (4,) = (32, 4)$, Simpan self.X = X untuk backward (z1:(32,4)) -> ReLU, $a_1 = \max(0, z_1)$, Simpan self.mask = $(z_1 > 0)$ untuk backward (a_1: (32, 4)) -> Linear(4, 3), $z_2 = a_1 @ W_2^T + b_2$, Shape: $(32,4) \times (4,3)^T + (3,) = (32, 3)$, Simpan self.X = a_1 untuk backward (z_2: (32, 3)) -> Softmax, $\hat{y} = \text{softmax}(z_2)$, Setiap baris menjadi distribusi probabilitas, Simpan self.output untuk backward ($\hat{y}$: (32, 3), probabilitas untuk 3 kelas)

2.1.10 Penjelasan Backward Propagation & Weight Update

Backward propagation menghitung gradien loss terhadap setiap parameter (bobot & bias) menggunakan chain rule. Gradien mengalir dari output layer ke input layer (kebalikan dari forward).

```
def backward(self, grad, lr, reg_type=None, reg_lambda=0.0, optimizer=None,
t=1):
    for layer in reversed(self.layers):         # iterasi terbalik
        if isinstance(layer, (Linear, RMSNorm)):
            grad = layer.backward(grad, lr, reg_type, reg_lambda, optimizer, t)
        else:
            grad = layer.backward(grad, lr)      # aktivasi tidak punya bobot
```

Melanjutkan contoh pada forward, maka backward nya akan:

Loss = CrossEntropy($\hat{y}$, y) -> $\partial L/\partial \hat{y} = (\hat{y} - \text{one_hot}(y)) / N$ (gradien awal dari loss) -> Softmax.backward, Hitung $\partial L/\partial z_2$ menggunakan Jacobian softmax ($\partial L/\partial z_2$: (32, 3)) -> Linear(4,3).backward, $\partial L/\partial W_2 = (\partial L/\partial z_2)^T @ a_1$ (gradien bobot), $\partial L/\partial b_2 = \Sigma(\partial L/\partial z_2, \text{axis}=0)$ (gradien bias), $\partial L/\partial a_1 = \partial L/\partial z_2 @ W_2$ (gradien ke layer sebelumnya), Update $W_2 \leftarrow W_2 - \eta \cdot \partial L/\partial W_2$ dan $b_2 \leftarrow b_2 - \eta \cdot \partial L/\partial b_2$ ($\partial L/\partial a_1$: (32, 4)) -> ReLU.backward, $\partial L/\partial z_1 = \partial L/\partial a_1$ * mask (mask = 1 dimana $z_1>0$, 0 lainnya) dan tidak ada parameter untuk diupdate ($\partial L/\partial z_1$: (32, 4)) -> Linear(4,4).backward, $\partial L/\partial W_1 = (\partial L/\partial z_1)^T @ X$ (gradien bobot), $\partial L/\partial b_1 = \Sigma(\partial L/\partial z_1, \text{axis}=0)$ (gradien bias), Update $W_1 \leftarrow W_1 - \eta \cdot \partial L/\partial W_1$ dan $b_1 \leftarrow b_1 - \eta \cdot \partial L/\partial b_1$.

Regularisasi pada Backward:

Jika regularisasi diaktifkan, gradien bobot ditambahkan penalti sebelum update:

- L1: dw += λ · sign(W) — mendorong bobot ke 0 (sparsity)
- L2: dw += 2λ · W — mendorong bobot tetap kecil (weight decay)

Regularisasi juga ditambahkan ke nilai loss:

- L1: loss += λ · Σ|W|
- L2: loss += λ · ΣW^2

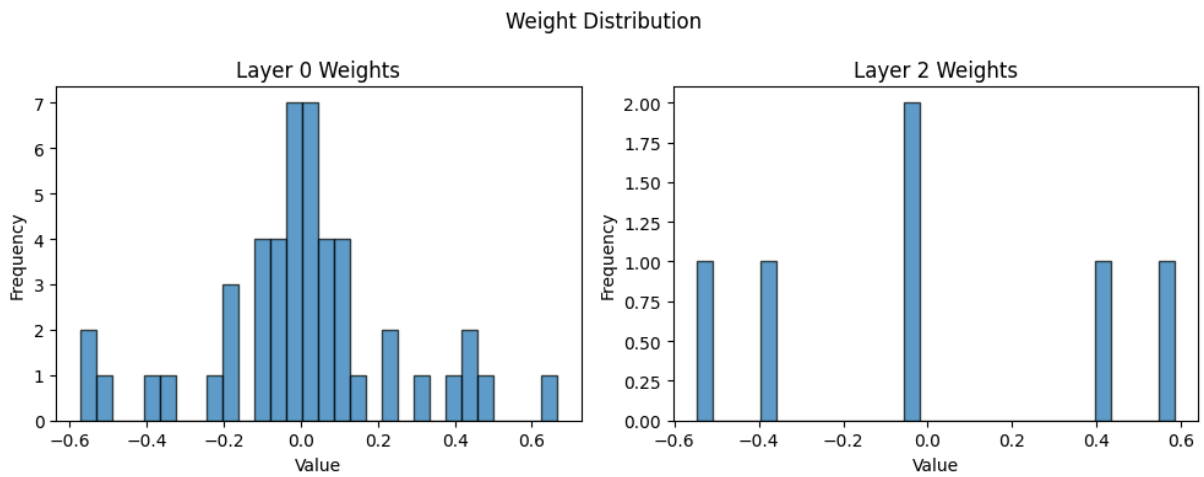
2.2 Hasil Pengujian

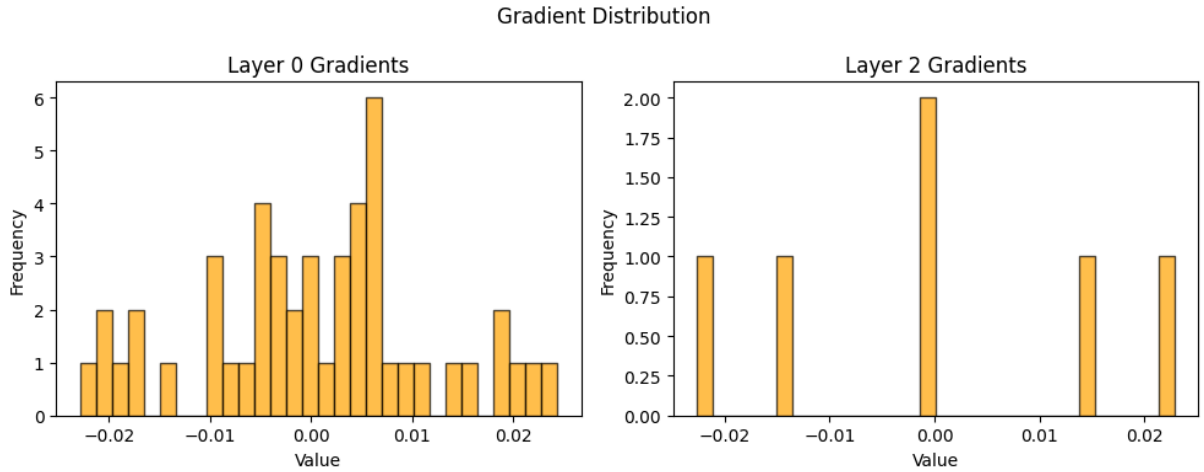
2.2.1 Pengaruh depth dan width

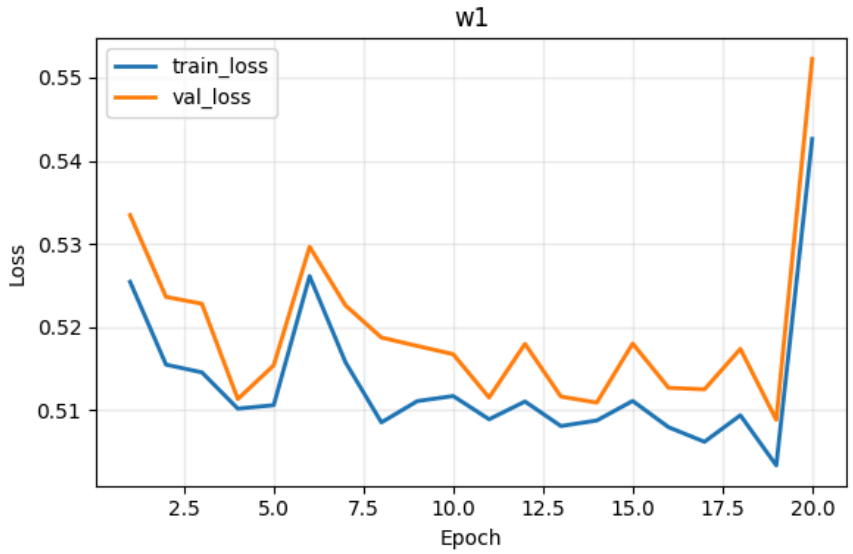
i. Perbandingan F1 Score

Model Variation	F1 Score
Control (3 layers: in, hidden (RELU), out)	0.7683664649956785
Width = 2	0.7665656128194024
Width = 8	0.7662505380972879
Width = 64	0.770618556701031
Depth = 2	0.7570921985815603
Depth = 4	0.7579135086937138
Depth = 8	0.6622039134912462

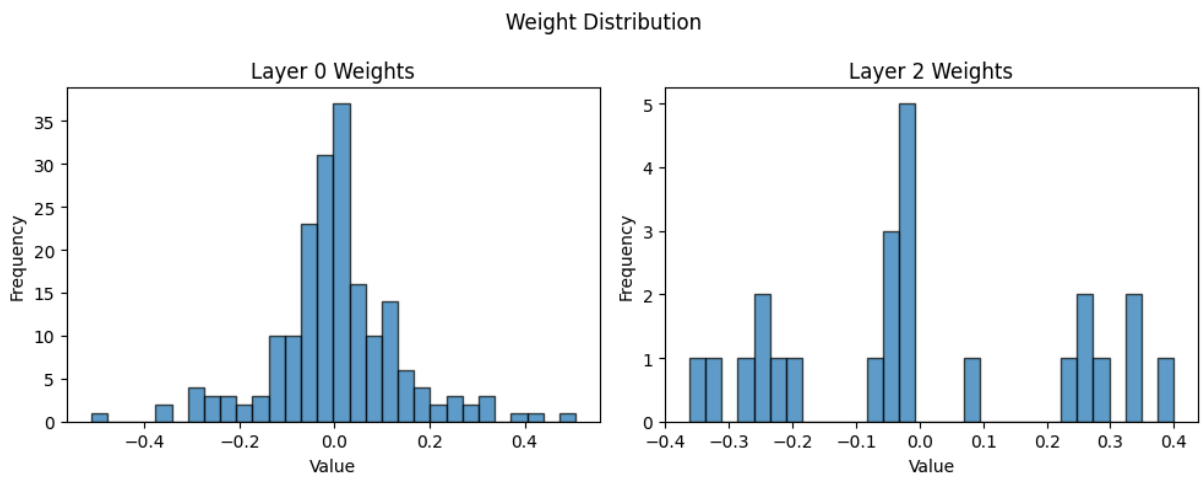
ii. Width = 2

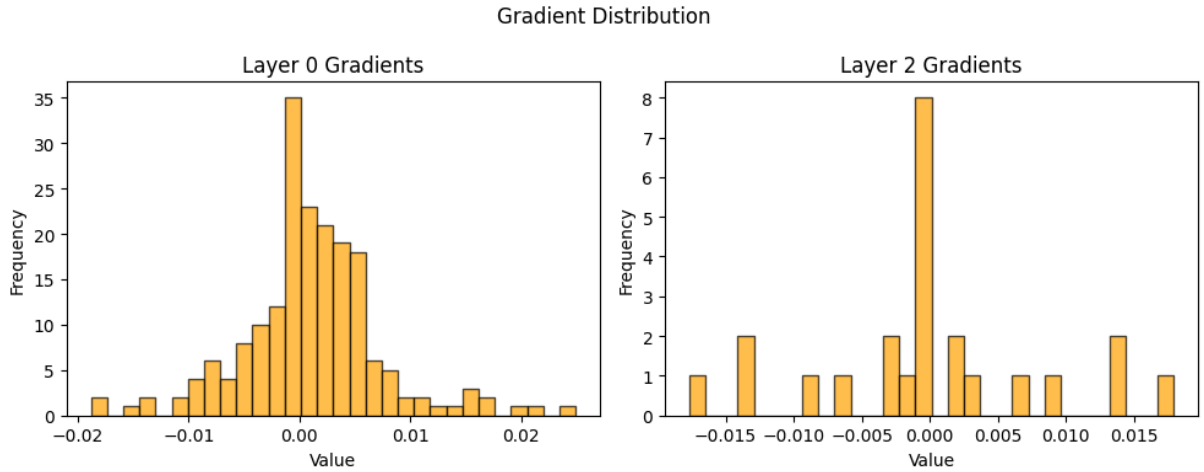



Gradient Distribution
Layer 0 Gradients
Layer 2 Gradients
Frequency
Value


w1
train_loss
val_loss
Loss
Epoch

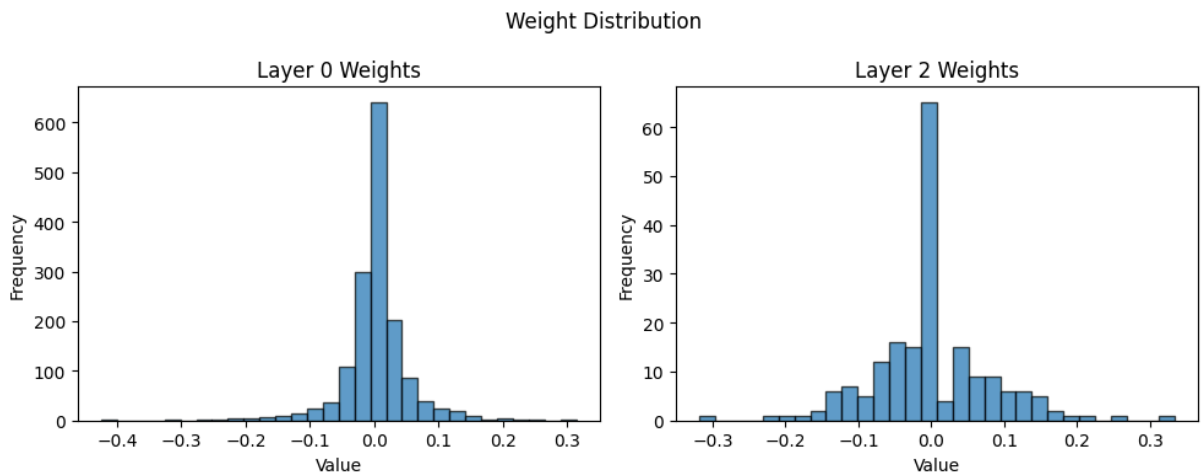
iii. Width = 8

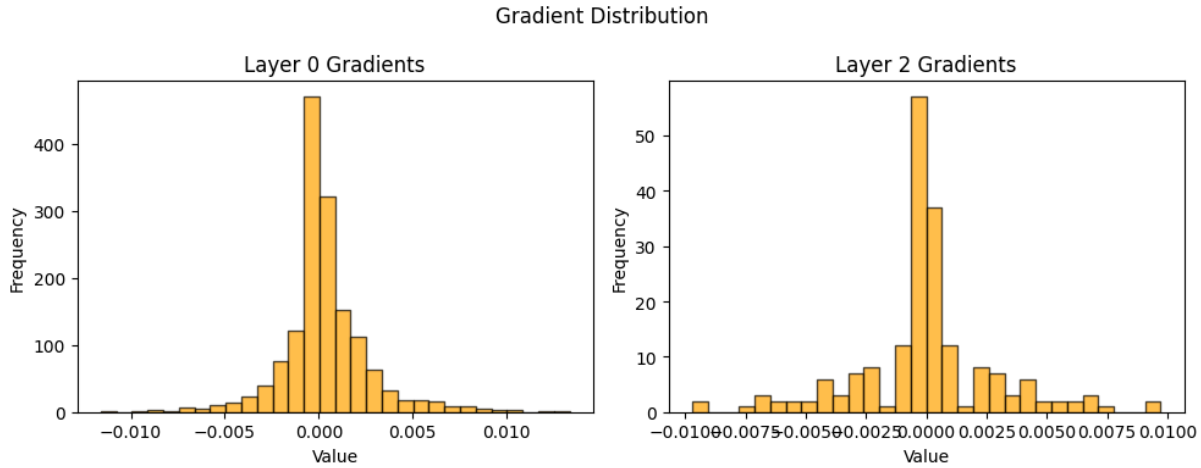

Weight Distribution
Layer 0 Weights
Layer 2 Weights
Frequency
Value

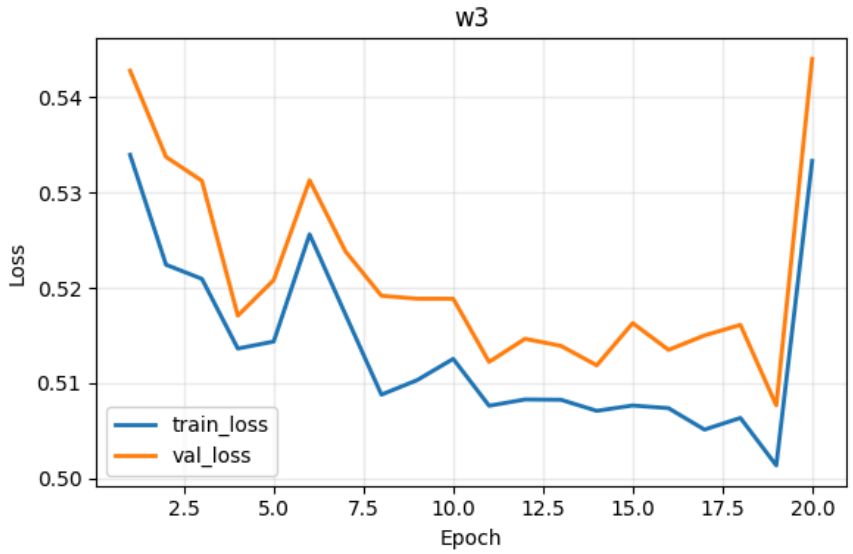

Gradient Distribution
Layer 0 Gradients
Layer 2 Gradients
Frequency
Value

w2
train_loss
val_loss
Loss
Epoch

iv. Width = 64

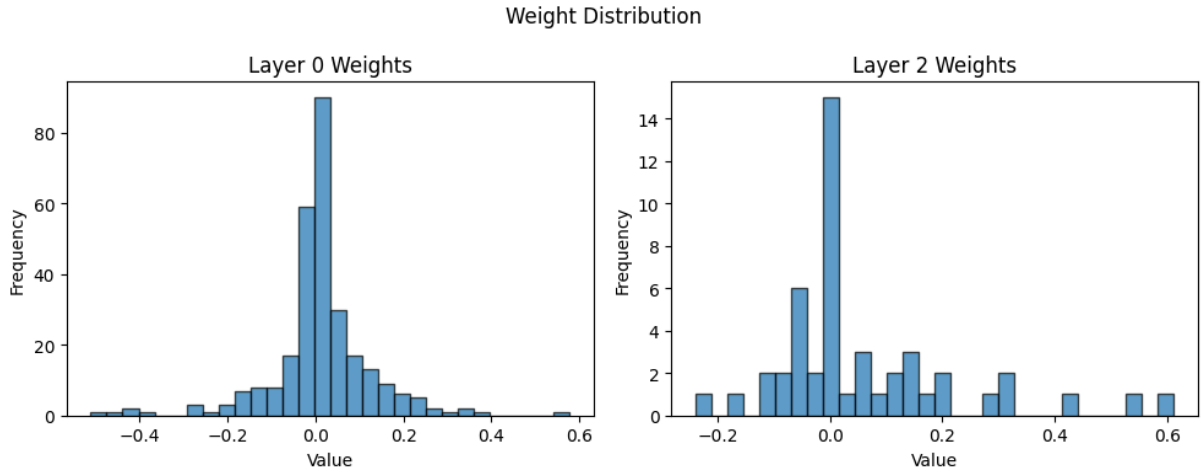

Weight Distribution
Layer 0 Weights
Layer 2 Weights
Frequency
Value

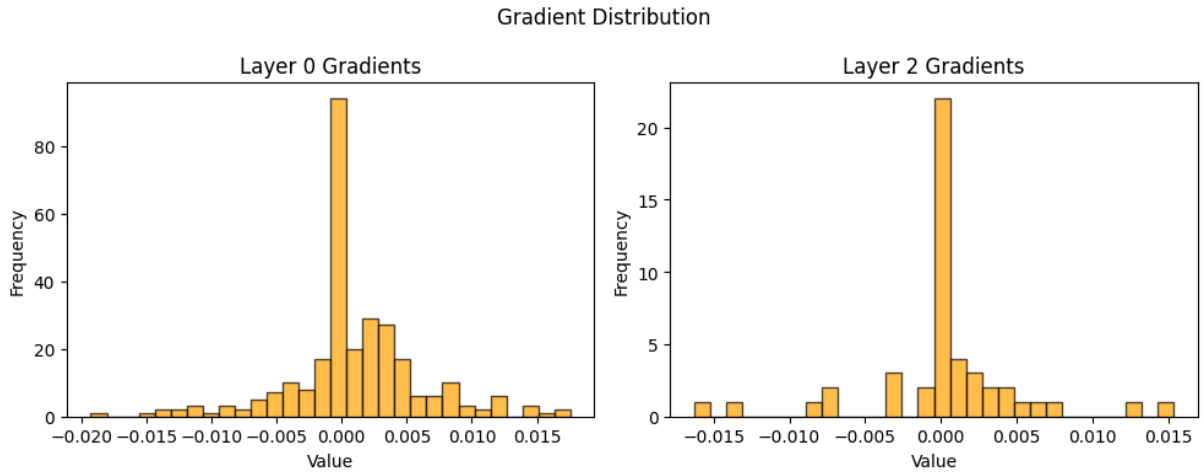


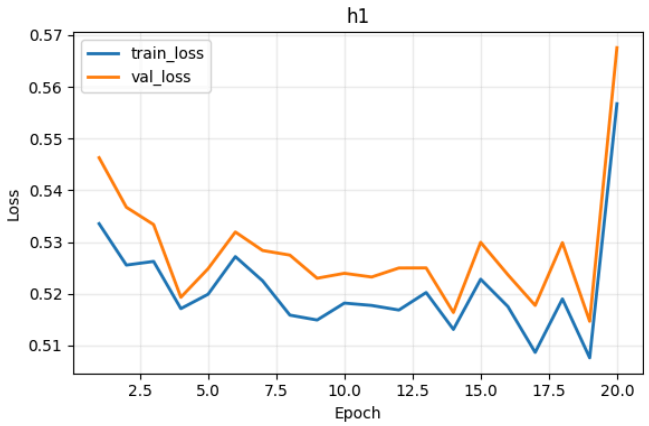


v. Depth = 2

```
h1_model = ffnn.Model(
    layers=[ffnn.Linear(24, 12),
            ffnn.Relu(),
            ffnn.Linear(12, 4),
            ffnn.Relu(),
            ffnn.Linear(4, 3),
            ffnn.Softmax()
            ],
    loss=ffnn.CrossEntropyLoss(),
)
```



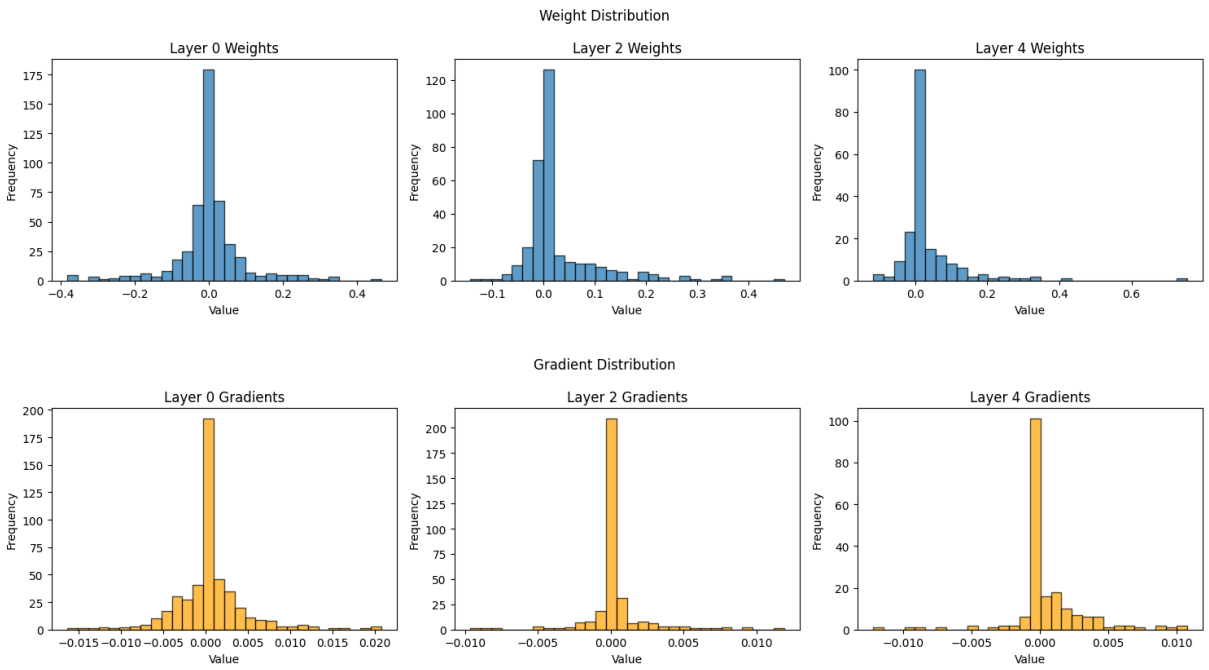


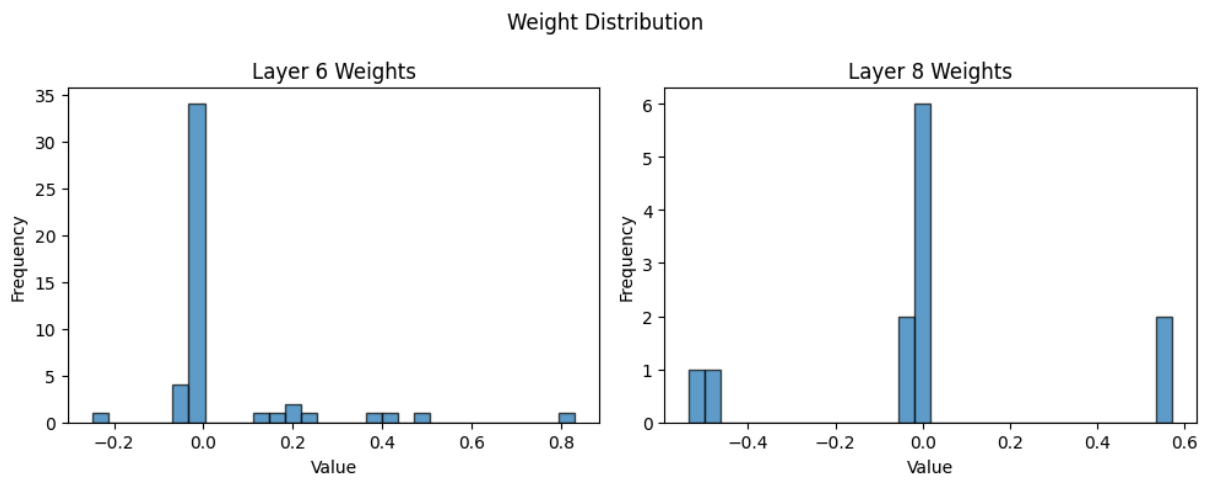


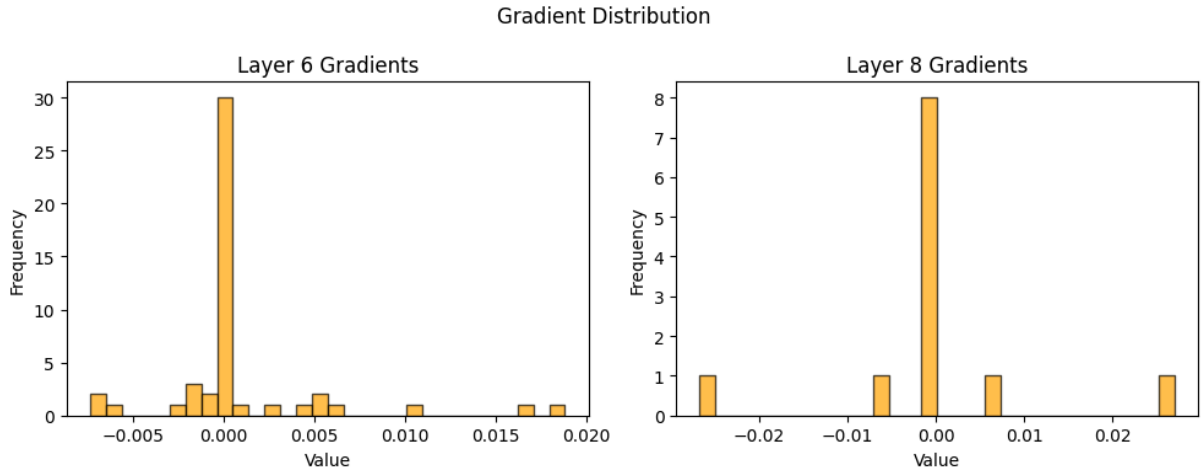
vi. Depth = 4

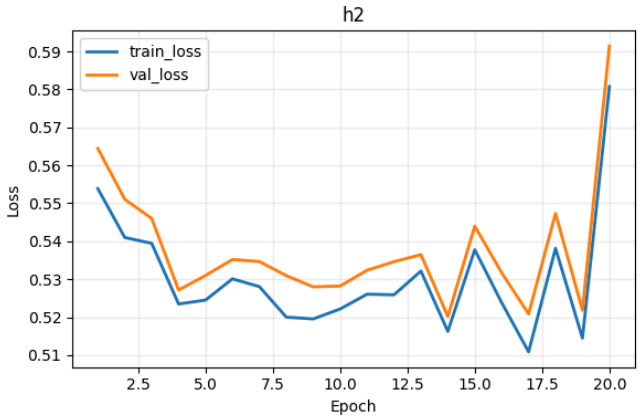
```
h2_model = ffnn.Model(
    layers=[ffnn.Linear(24, 20),
            ffnn.Relu(),
            ffnn.Linear(20, 16),
            ffnn.Relu(),
```

```
            ffnn.Linear(16, 12),
            ffnn.Relu(),
            ffnn.Linear(12, 4),
            ffnn.Relu(),
            ffnn.Linear(4, 3),
            ffnn.Softmax()
            ],
    loss=ffnn.CrossEntropyLoss(),
)
```





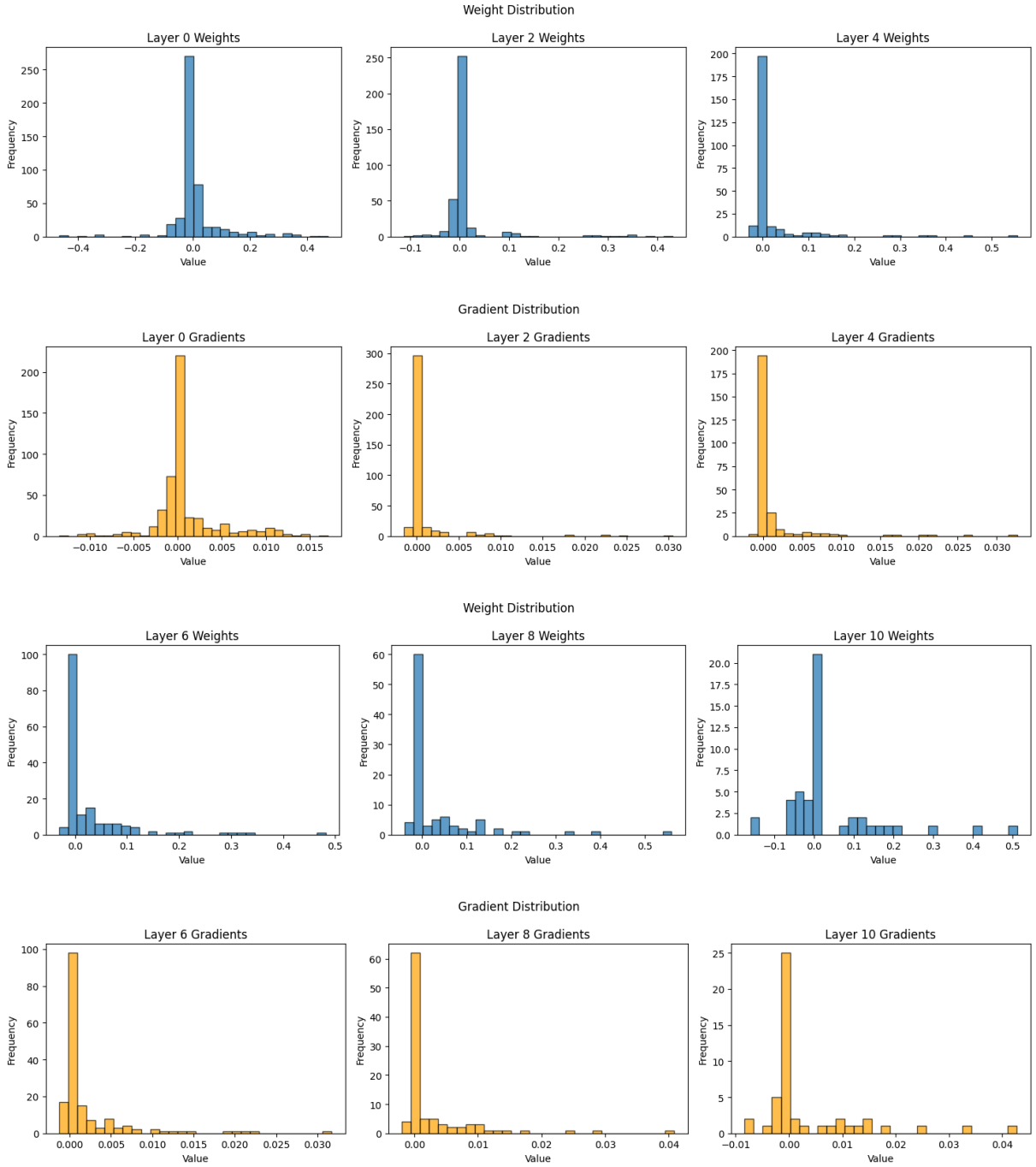


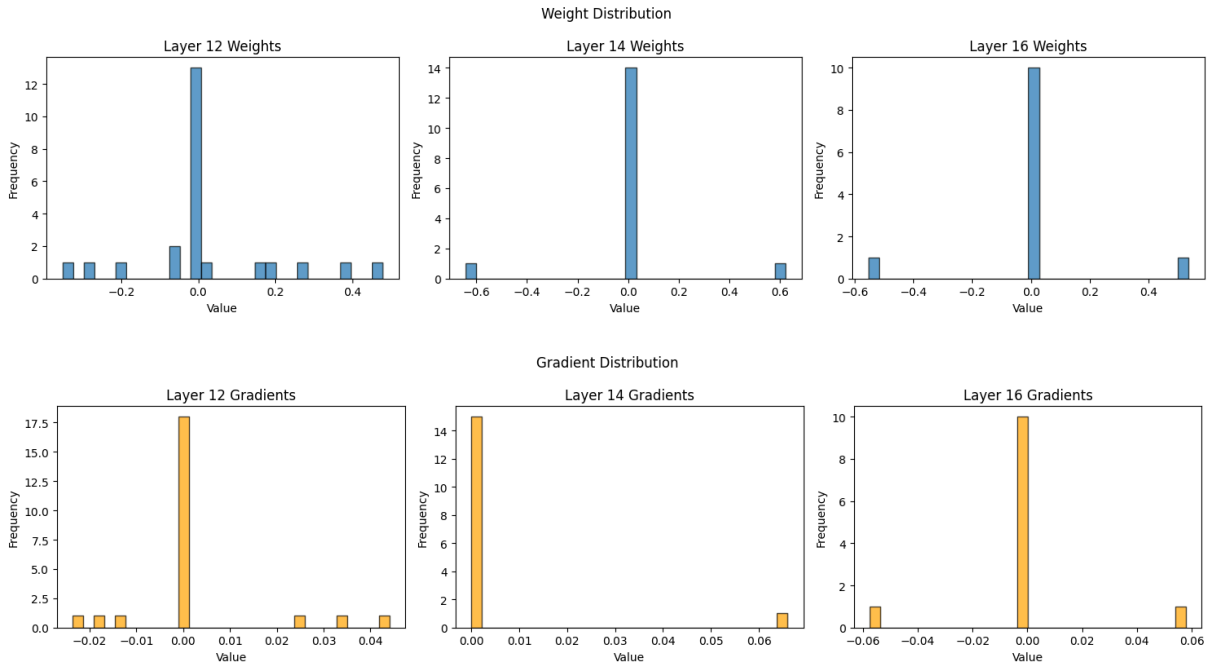


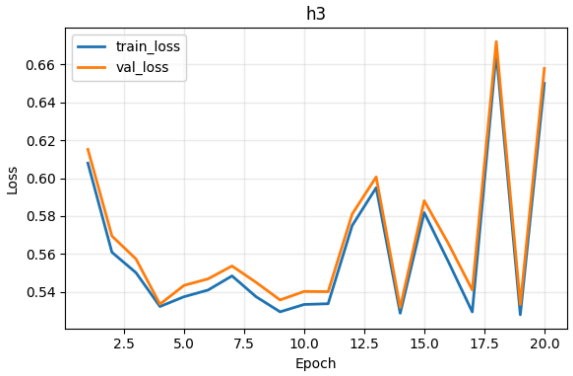
vii. Depth = 8

```
h3_model = ffnn.Model(
    layers=[ffnn.Linear(24, 20),
            ffnn.Relu(),
            ffnn.Linear(20, 18),
            ffnn.Relu(),
            ffnn.Linear(18, 14),
            ffnn.Relu(),
            ffnn.Linear(14, 12),
            ffnn.Relu(),
            ffnn.Linear(12, 8),
            ffnn.Relu(),
            ffnn.Linear(8, 6),
            ffnn.Relu(),
            ffnn.Linear(6, 14),
            ffnn.Relu(),
            ffnn.Linear(4, 4),
```

```
            ffnn.Relu(),
            ffnn.Linear(4, 3),
            ffnn.Softmax()
            ],
    loss=ffnn.CrossEntropyLoss(),
)
```





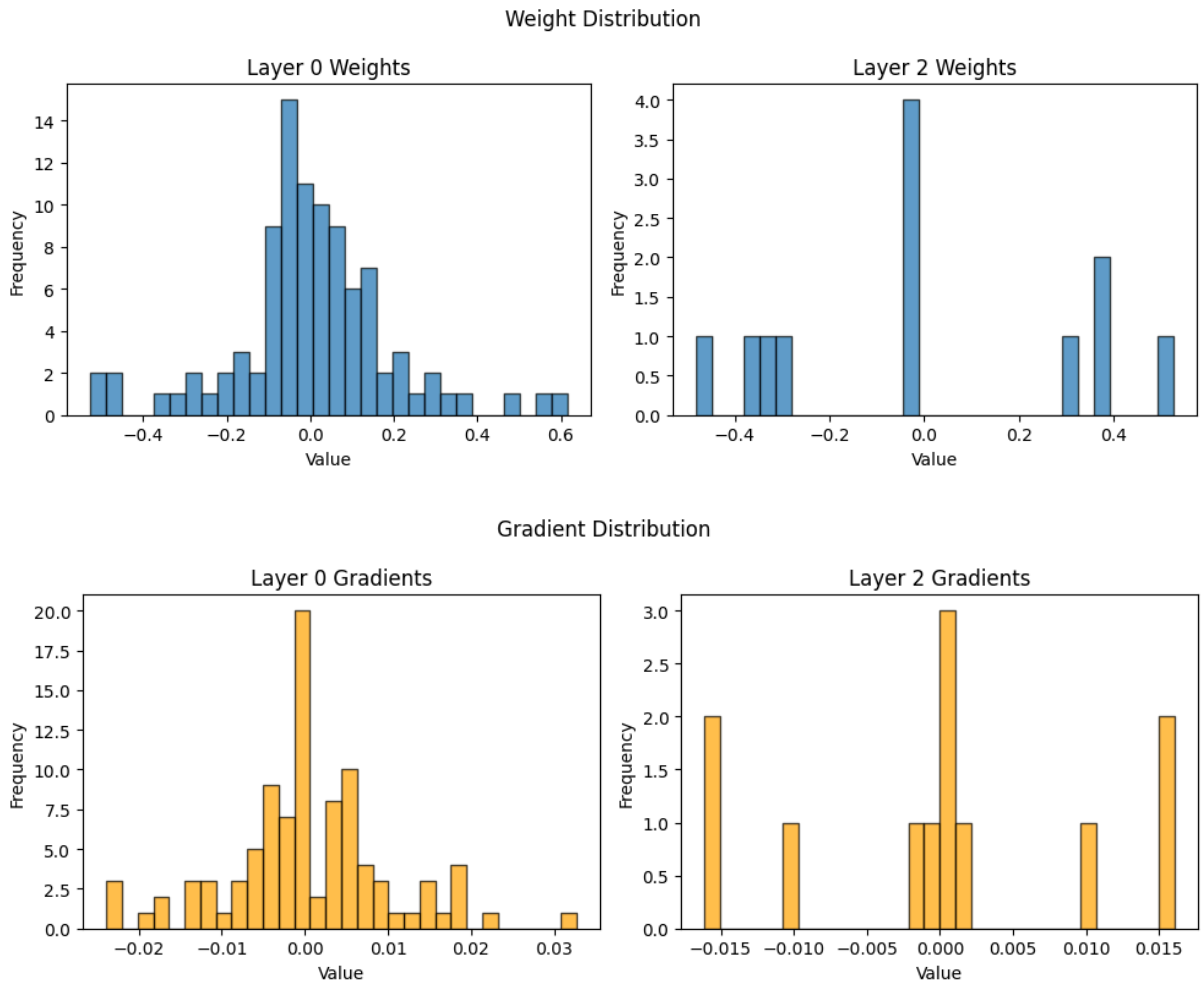


2.2.2 Pengaruh fungsi aktivasi

i. Perbandingan F1 Score

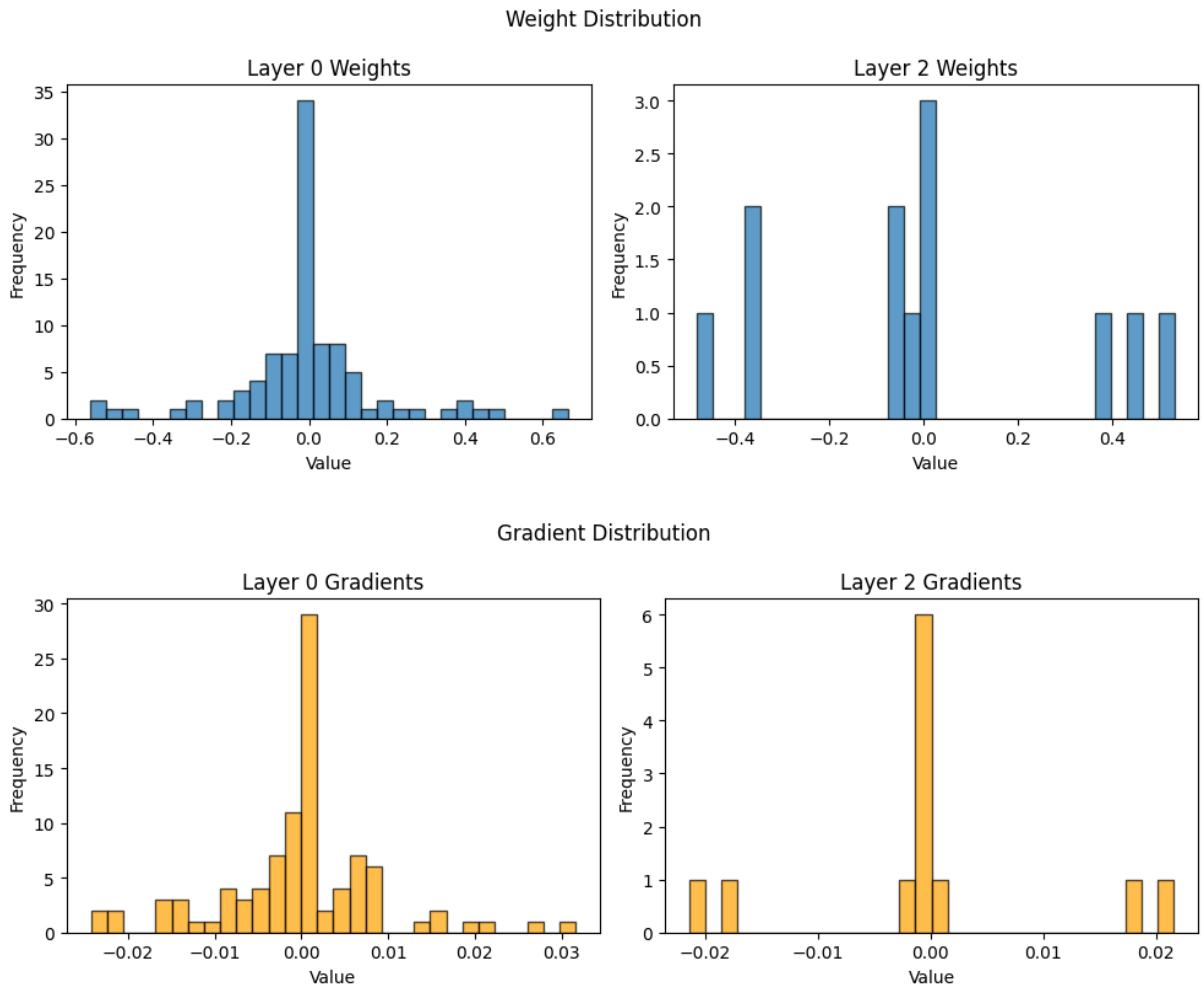
Model Variation	F1 Score
Control (3 layers: in, hidden (RELU), out)	0.7683664649956785
RELU	0.7636047017849369
Leaky RELU	0.7723367697594502
ELU	0.7736976942783945
RMS Norm	0.7726495726495727
Tanh	0.7705553164012053
Sigmoid	0.7696267696267697

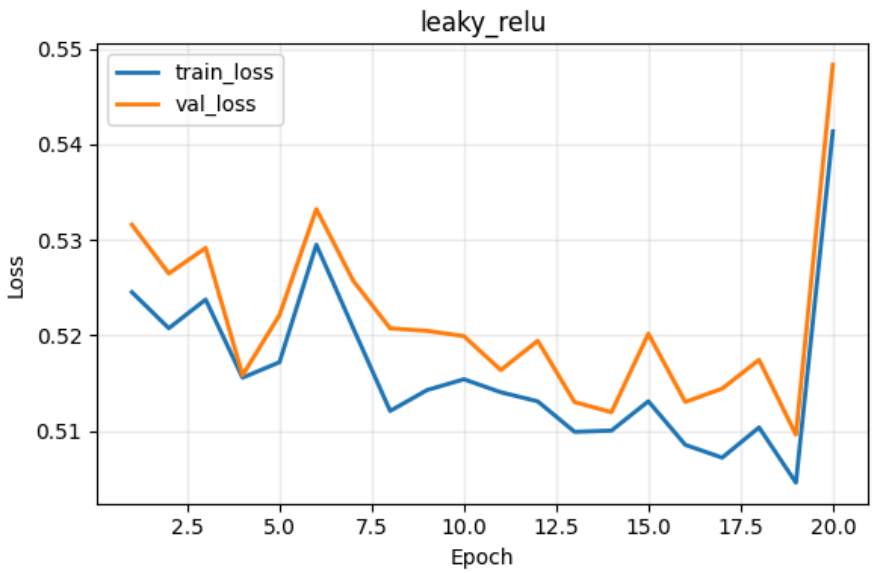
ii. RELU


Weight Distribution
Layer 0 Weights
Layer 2 Weights
Frequency
Value
Gradient Distribution
Layer 0 Gradients
Layer 2 Gradients
Frequency
Value

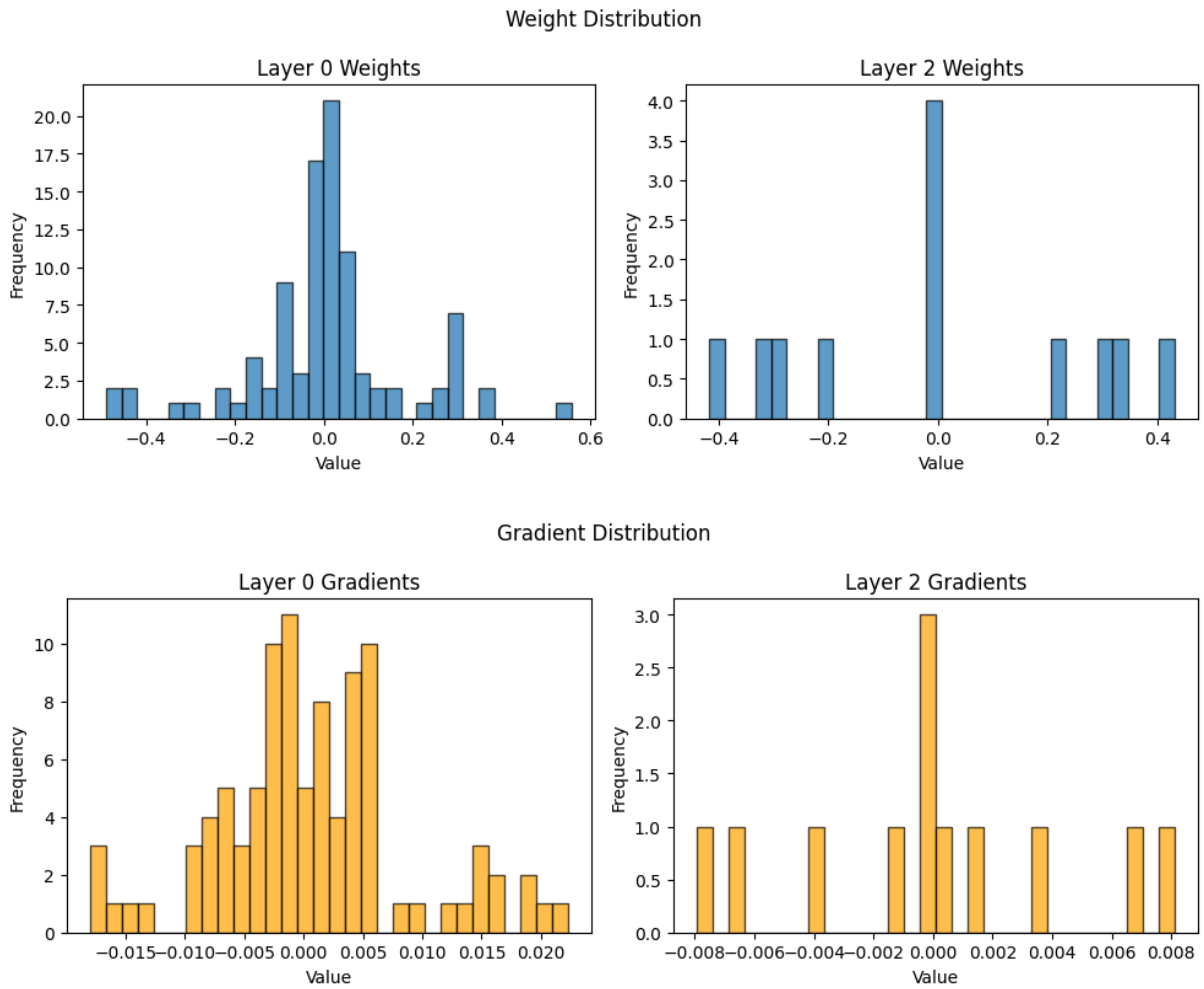
relu
train_loss
val_loss
Loss
Epoch

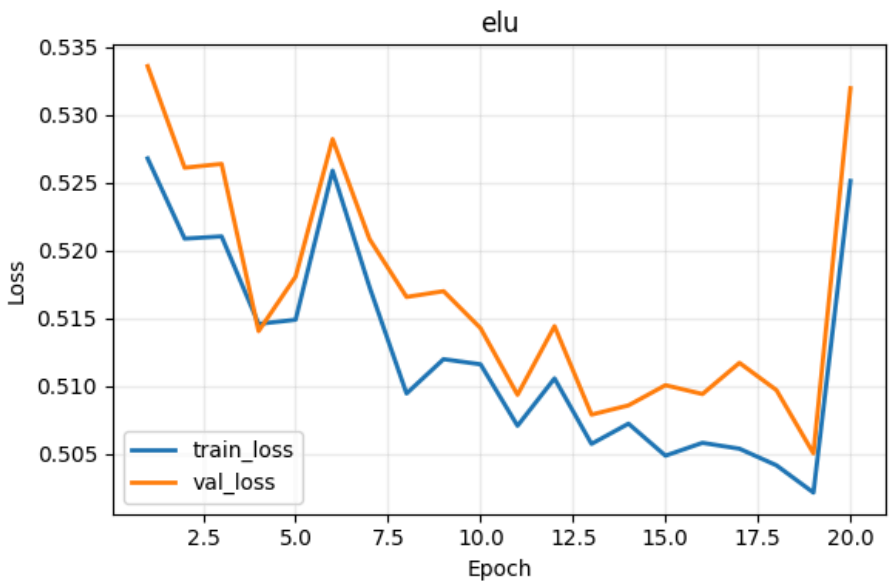
iii. Leaky RELU



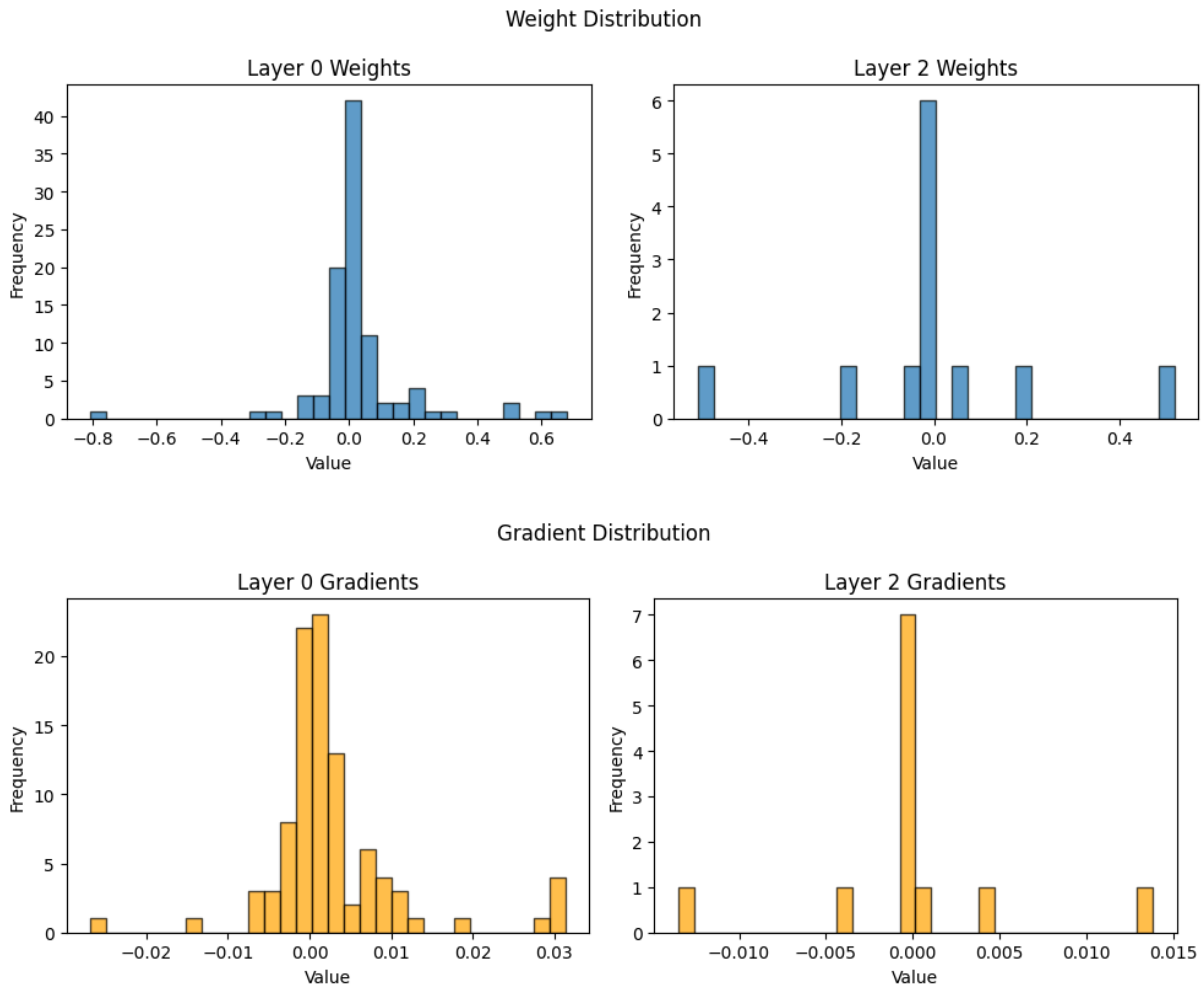


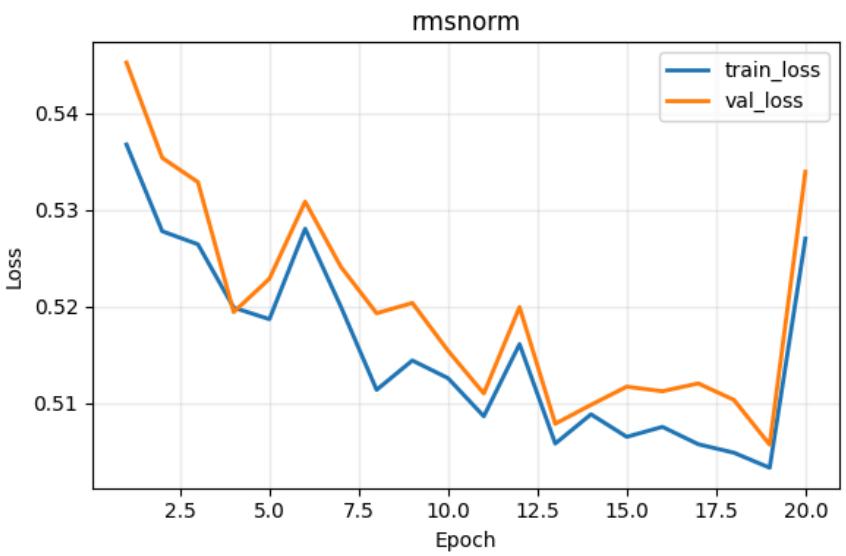
iv. ELU


Weight Distribution
Layer 0 Weights
Layer 2 Weights
Frequency
Value
Gradient Distribution
Layer 0 Gradients
Layer 2 Gradients
Frequency
Value

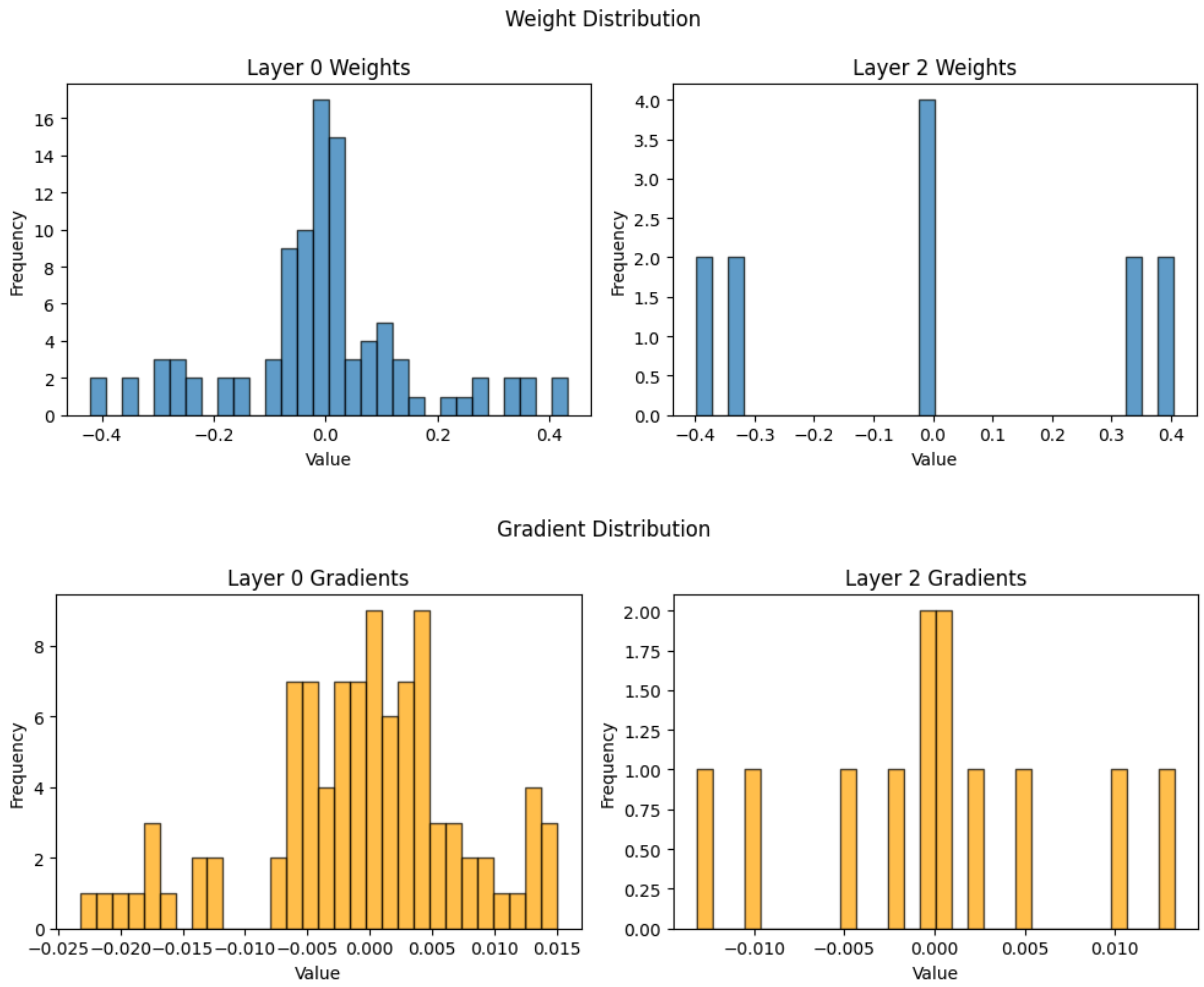

elu
Loss
Epoch
train_loss
val_loss

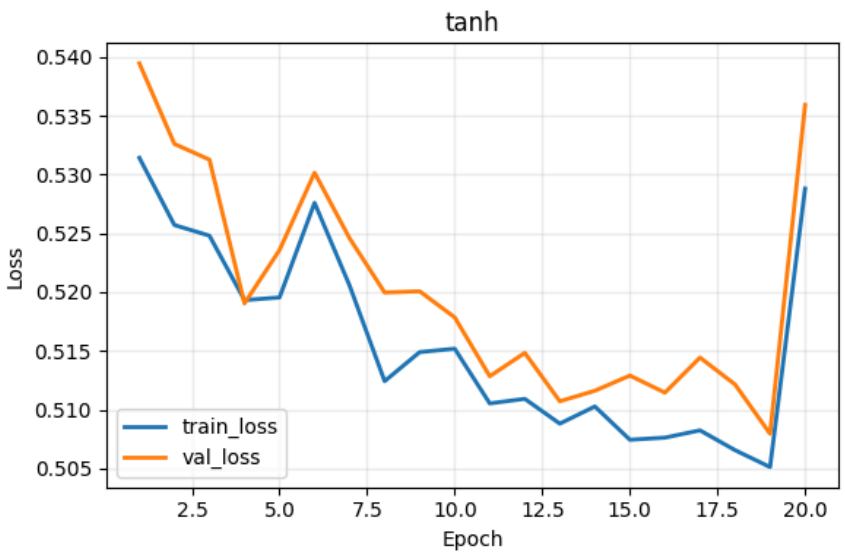
v. RMS Norm


Weight Distribution
Layer 0 Weights
Layer 2 Weights
Frequency
Value
Gradient Distribution
Layer 0 Gradients
Layer 2 Gradients
Frequency
Value

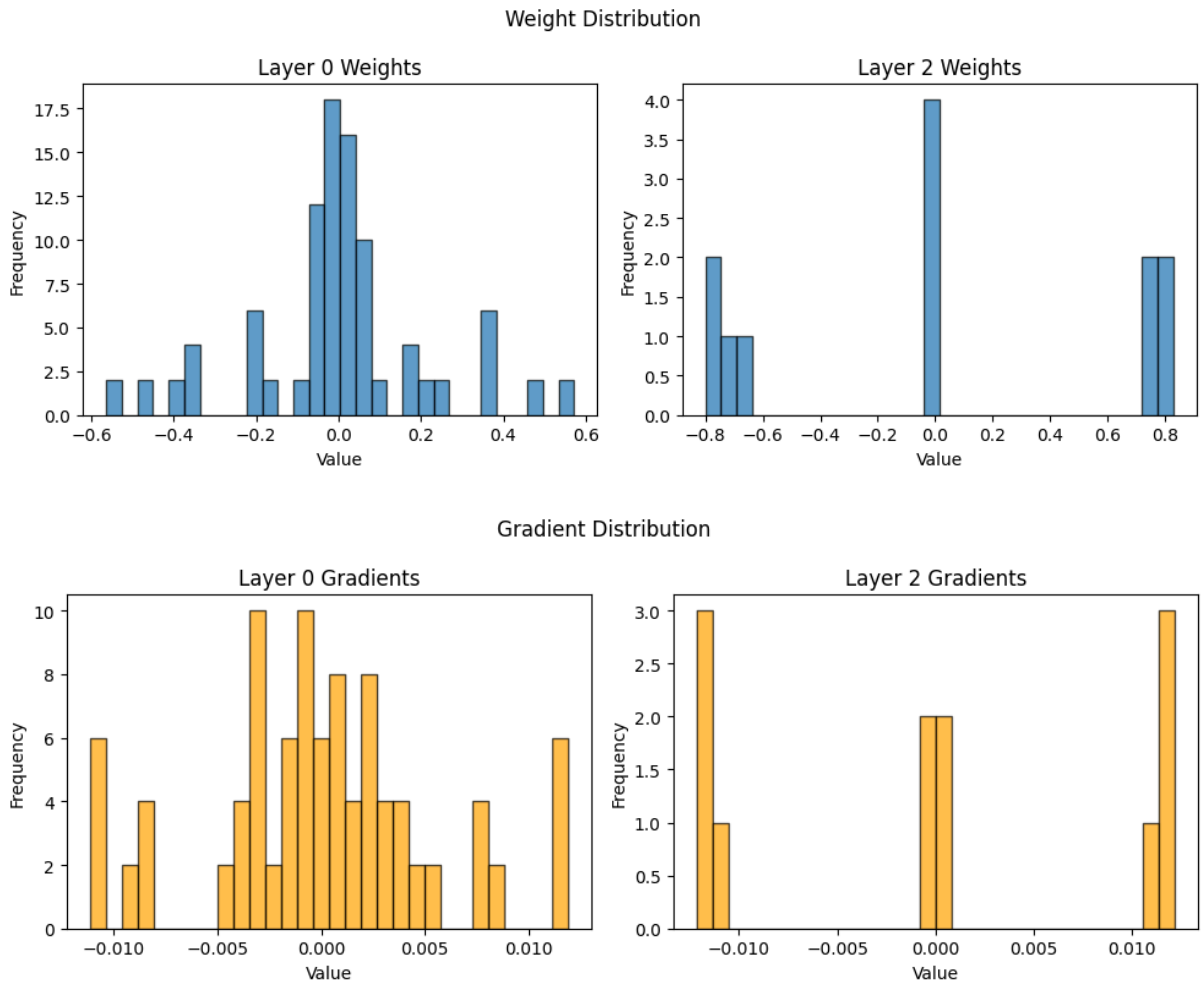

rmsnorm
train_loss
val_loss
Loss
Epoch

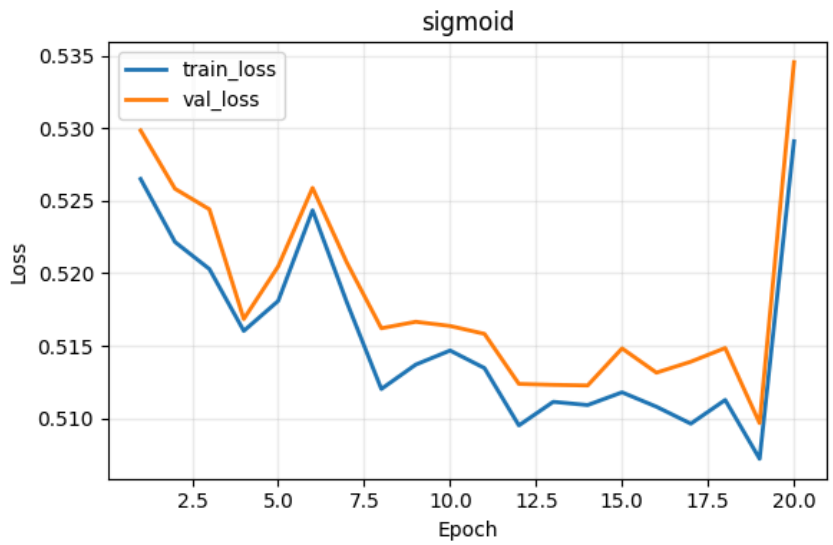
vi. Tanh


Weight Distribution
Layer 0 Weights
Layer 2 Weights
Frequency
Value
Gradient Distribution
Layer 0 Gradients
Layer 2 Gradients
Frequency
Value


tanh
Loss
Epoch
train_loss
val_loss

vii. Sigmoid


Weight Distribution
Layer 0 Weights
Layer 2 Weights
Frequency
Value
Gradient Distribution
Layer 0 Gradients
Layer 2 Gradients
Frequency
Value

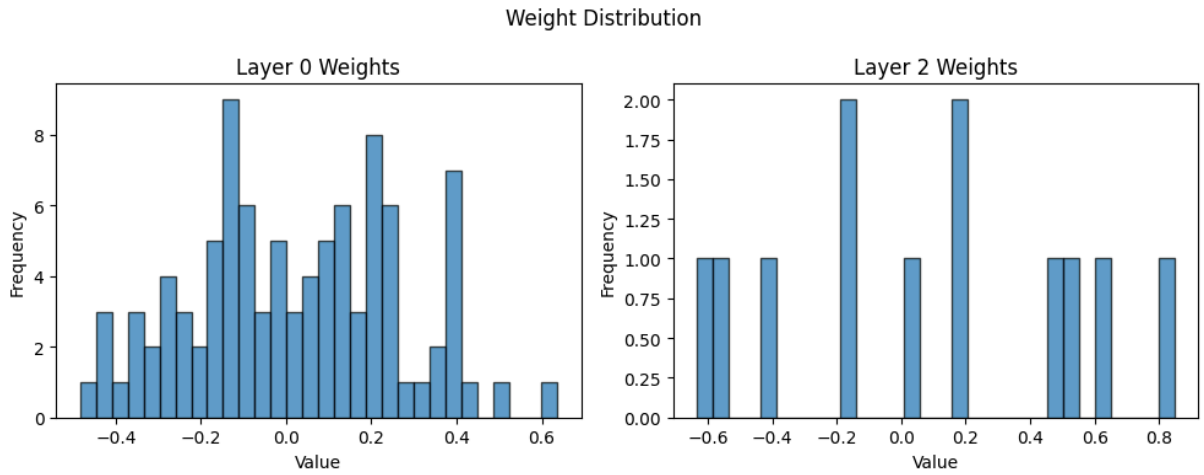

sigmoid
train_loss
val_loss
Loss
Epoch

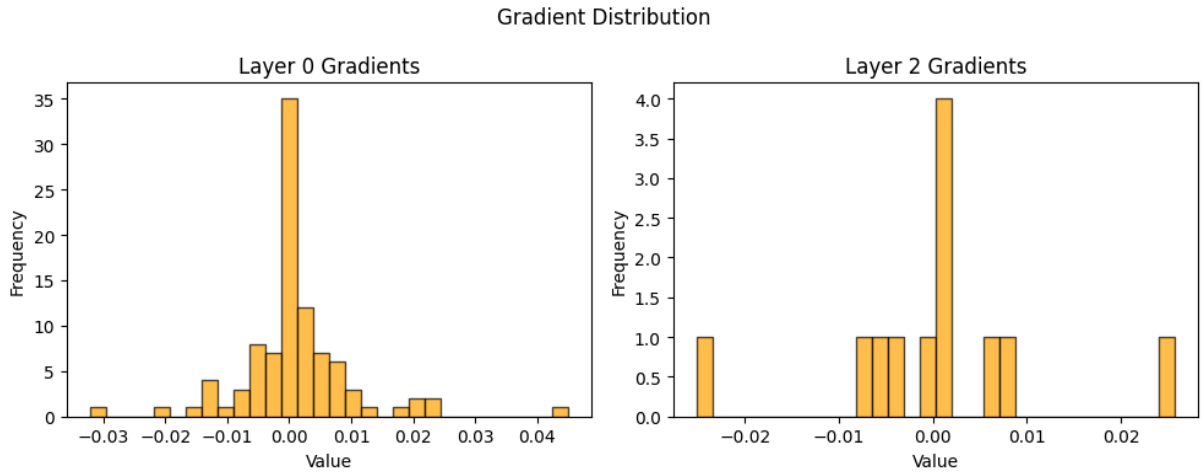
2.2.3 Pengaruh learning rate

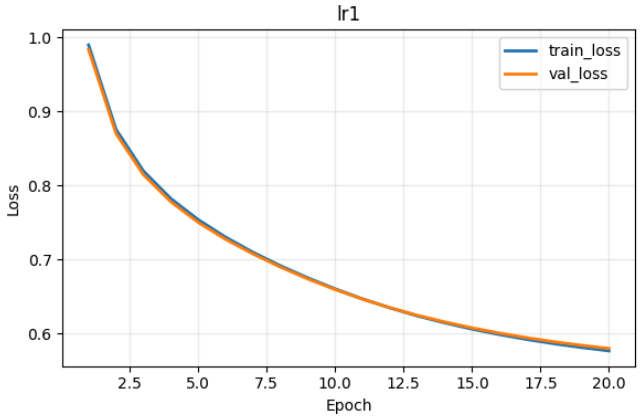
i. Perbandingan F1 Score

Model Variation	F1 Score
Control (3 layers: in, RELU, out, Lr=1)	0.7698924731182796
Lr = 0.01	0.8025127601099332
Lr = 0.1	0.7982207844723008
Lr = 1.1	0.7694300518134715

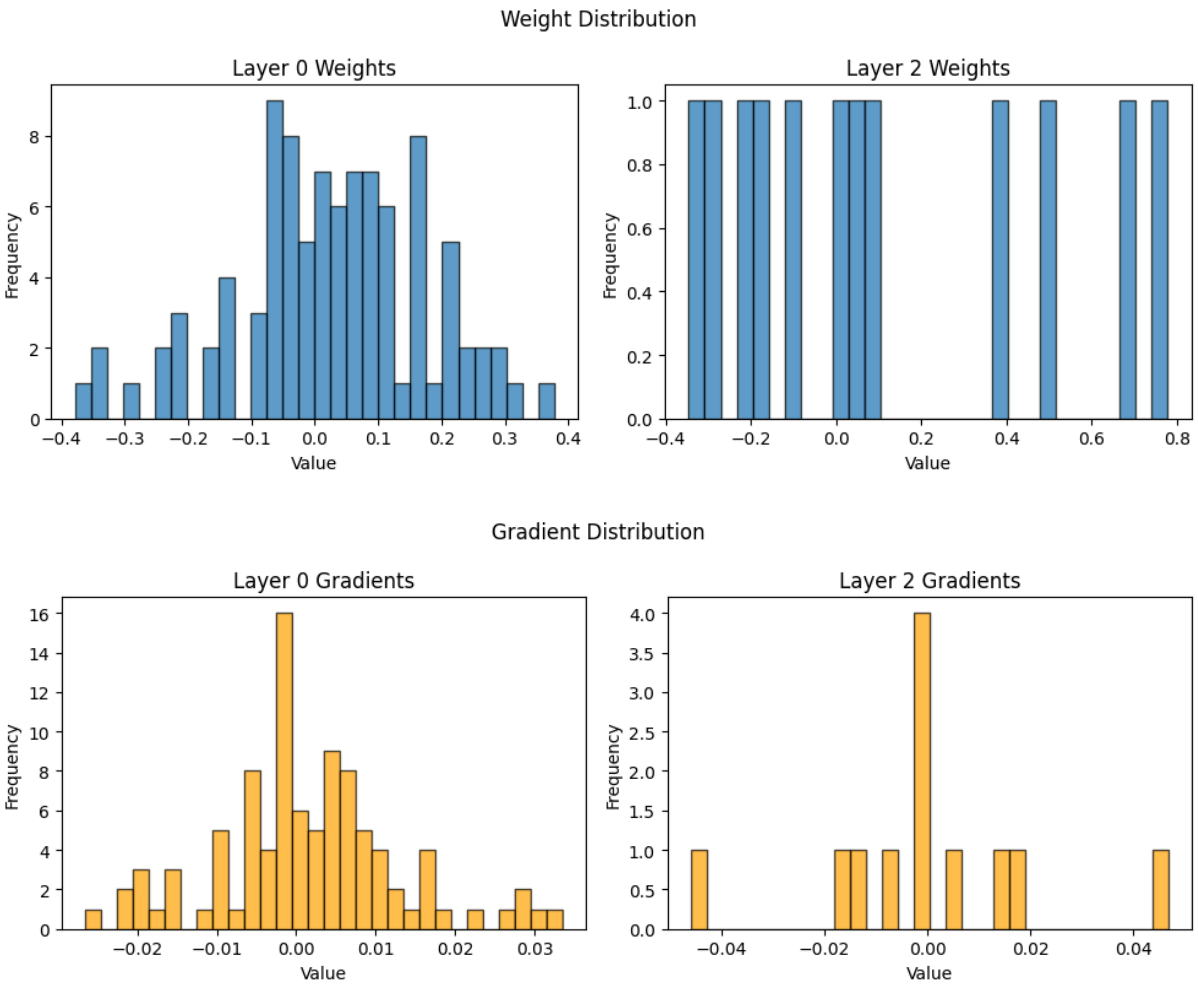
ii. Lr = 0.01

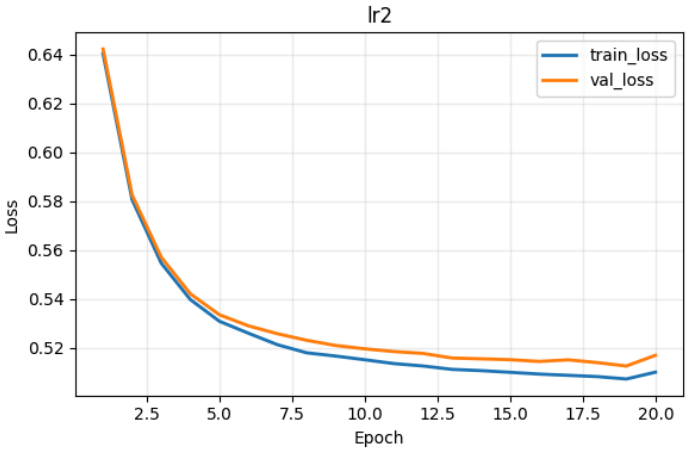




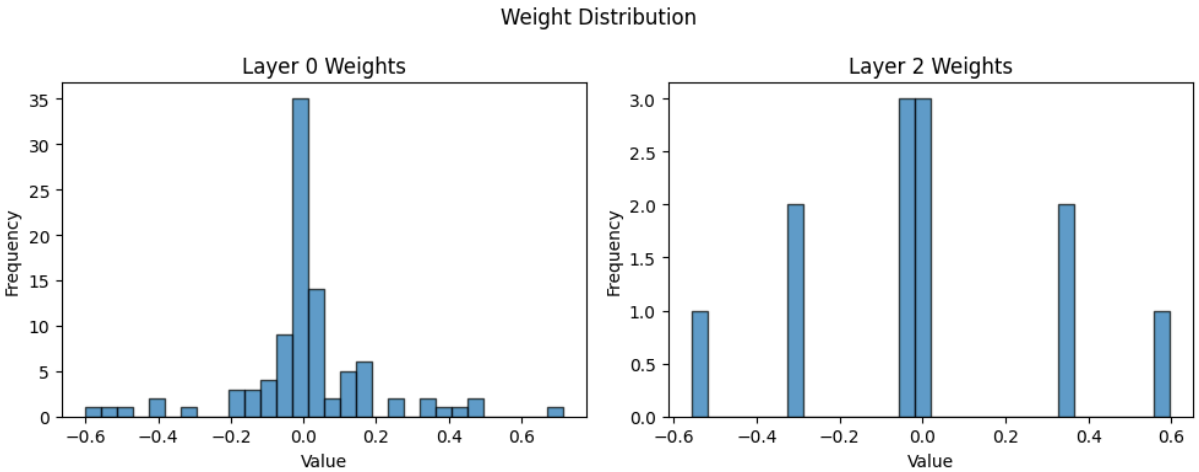

lr1
train_loss
val_loss
Loss
Epoch

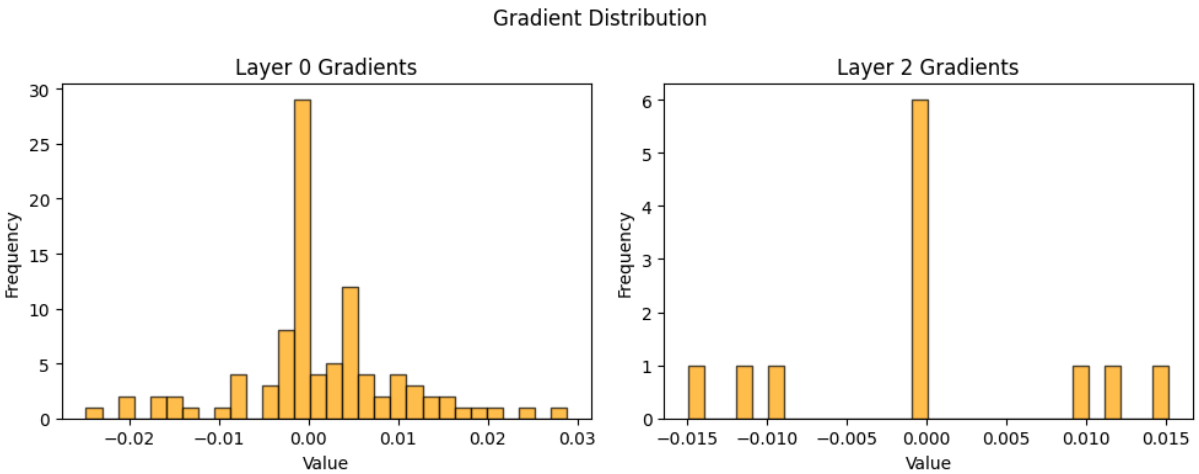
iii. Lr = 0.1

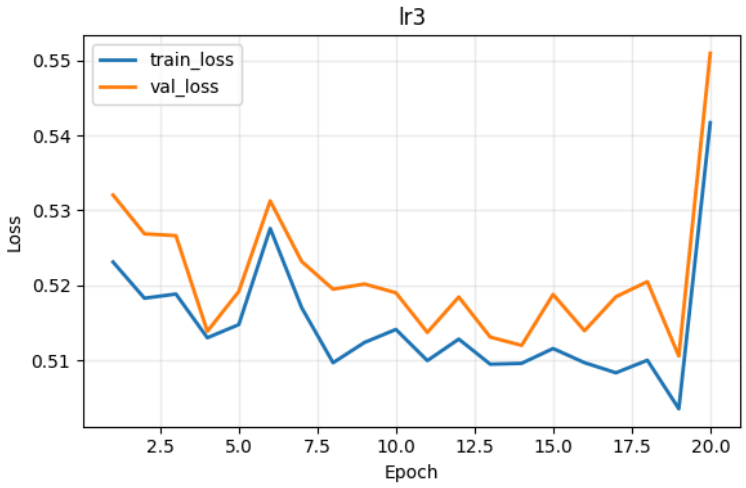

Weight Distribution
Layer 0 Weights
Layer 2 Weights
Frequency
Value
Gradient Distribution
Layer 0 Gradients
Layer 2 Gradients
Frequency
Value


lr2
train_loss
val_loss
Loss
Epoch
0.64
0.62
0.60
0.58
0.56
0.54
0.52
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

iv. Lr = 1.1


Weight Distribution
Layer 0 Weights
Layer 2 Weights
Frequency
Value

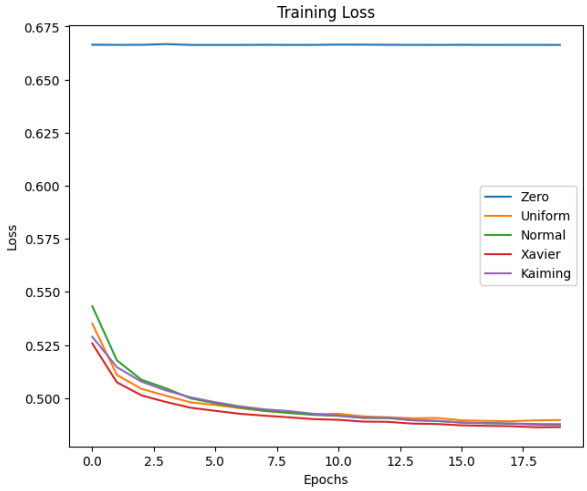

Gradient Distribution
Layer 0 Gradients
Layer 2 Gradients
Frequency
Value

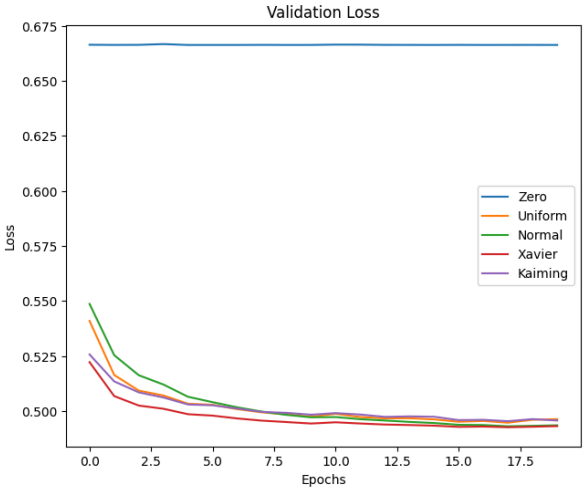


2.2.4 Pengaruh inisialisasi bobot

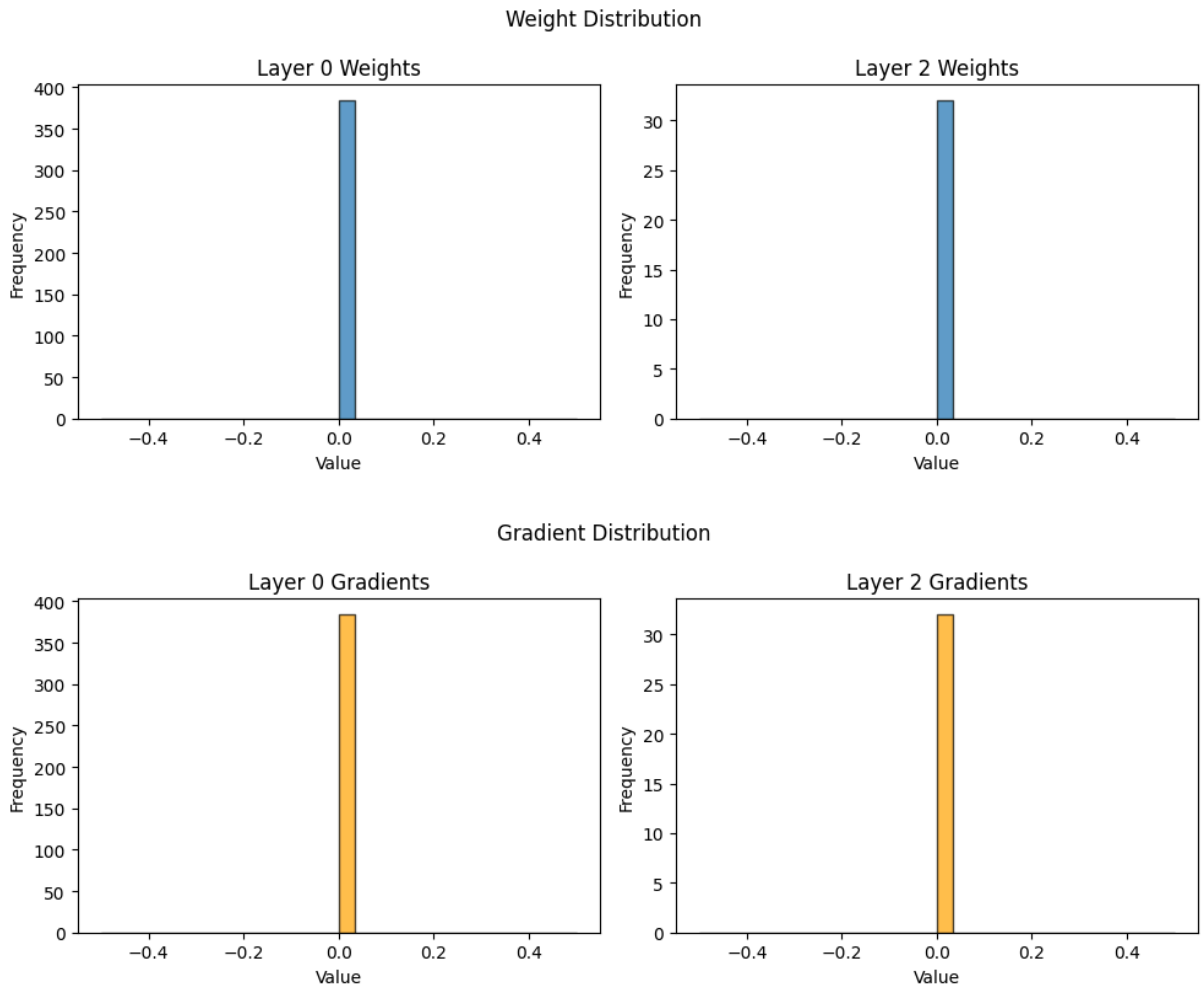
i. Perbandingan F1 Score

Model Variation	F1 Score
Zero	0.7620
Uniform	0.8029
Normal	0.8076
Xavier Uniform	0.8005
Kaiming Uniform (He)	0.8016

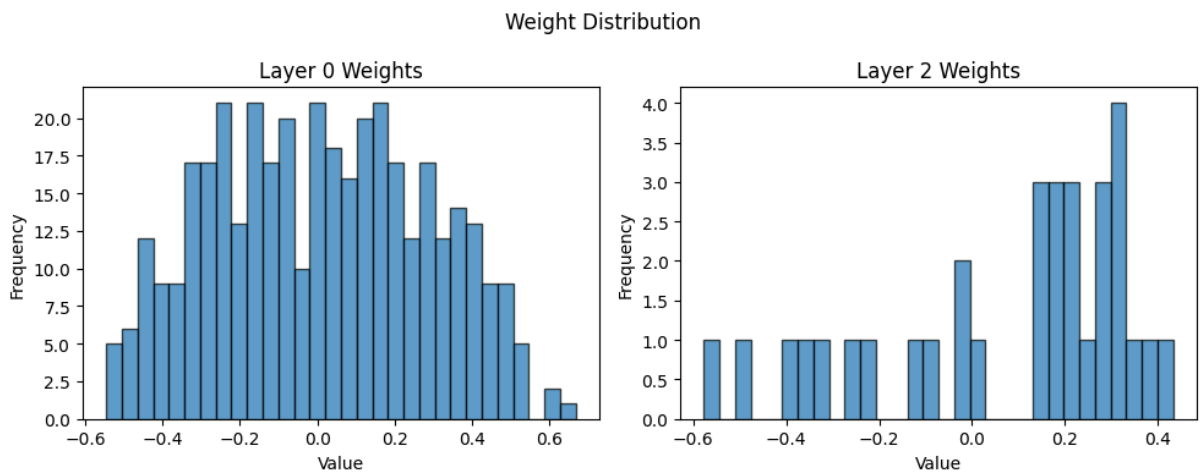


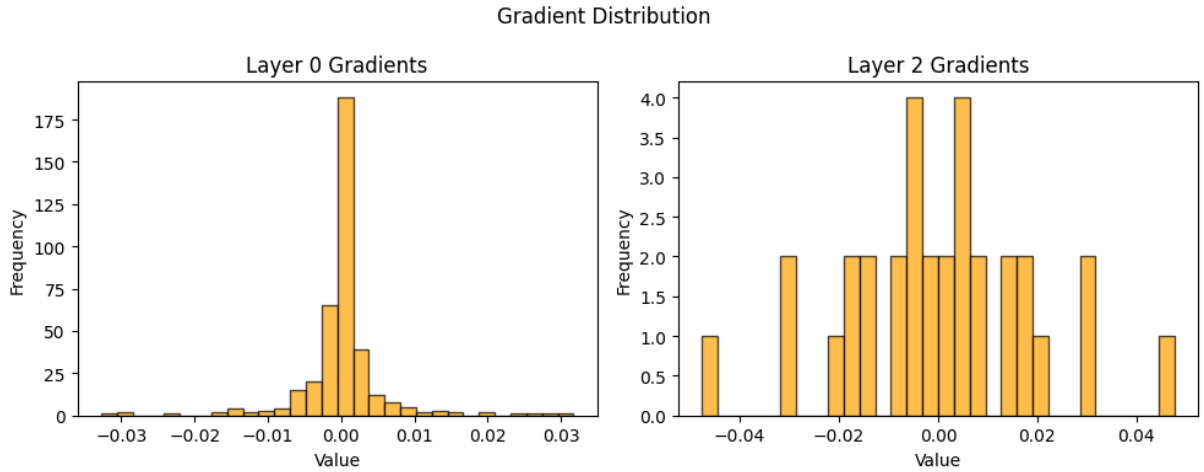


ii. Zero

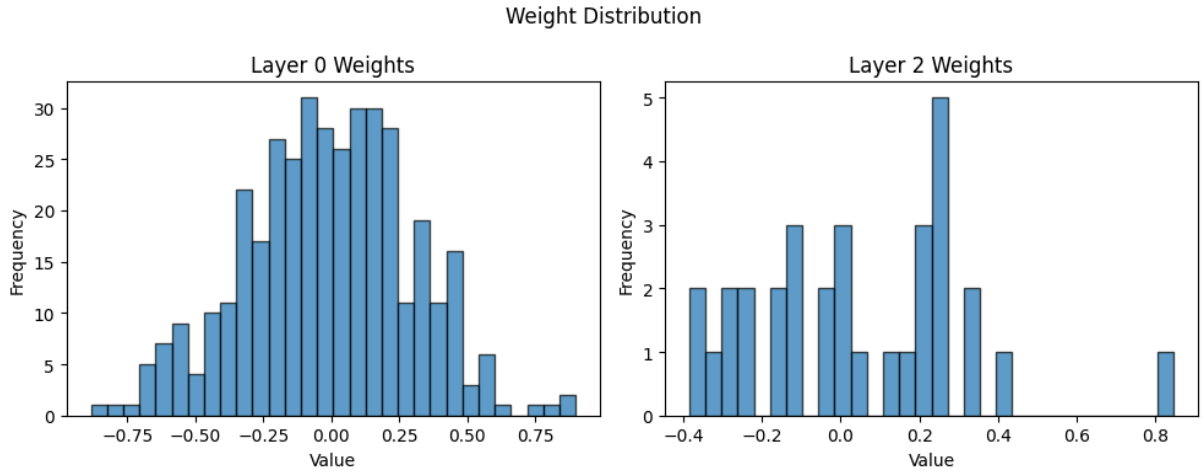


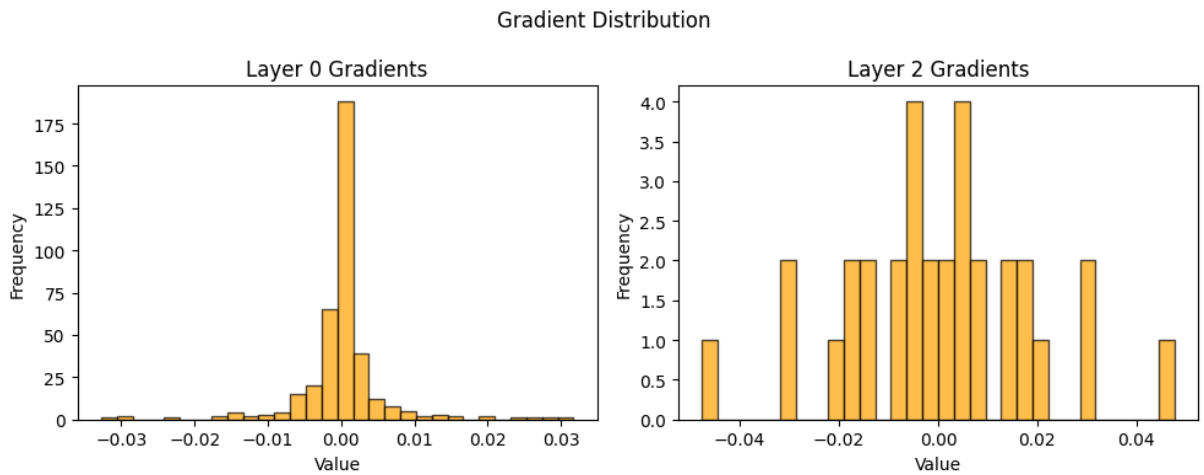
iii. Uniform



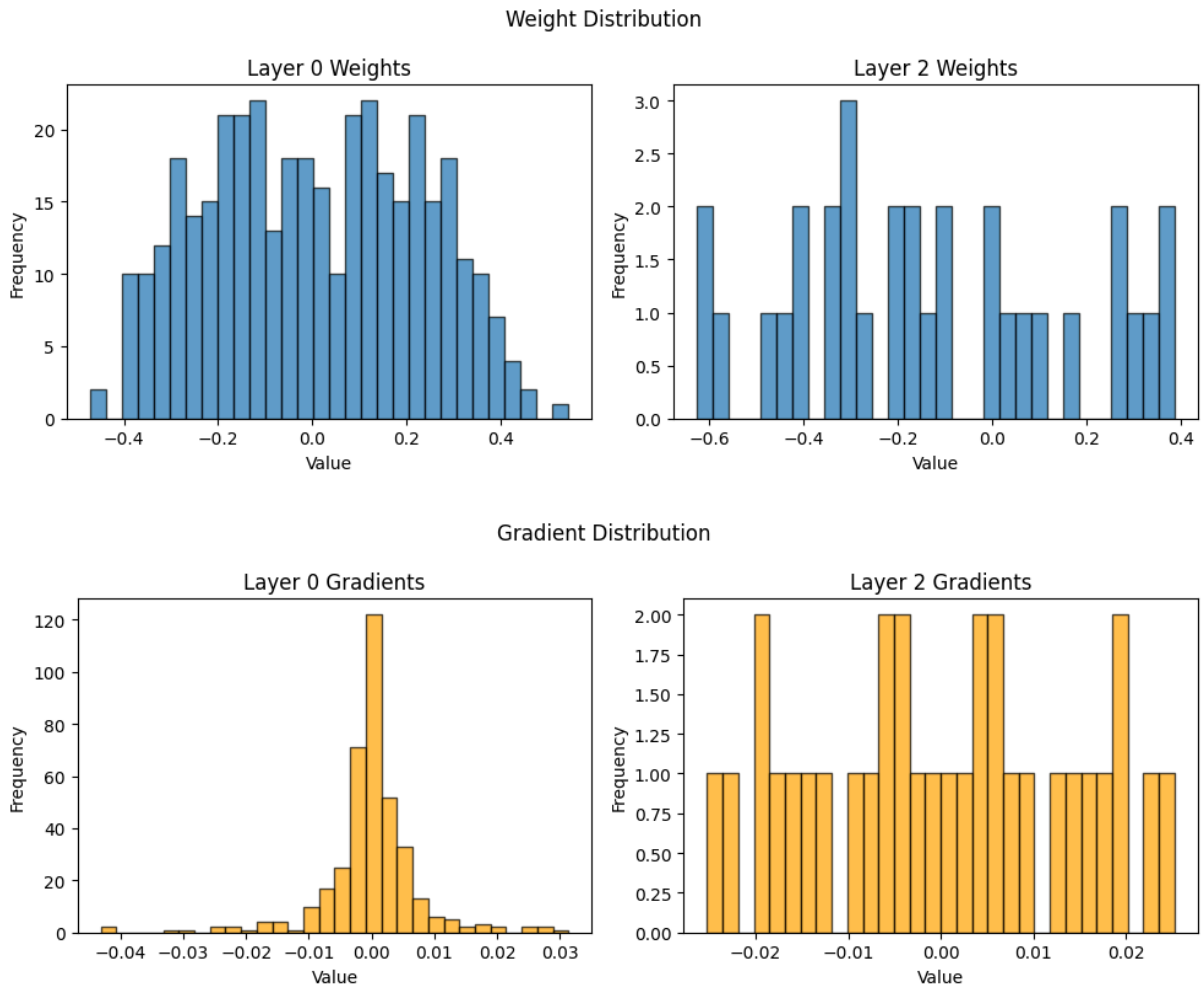

Gradient Distribution
Layer 0 Gradients
Layer 2 Gradients
Frequency
Value

iv. Normal

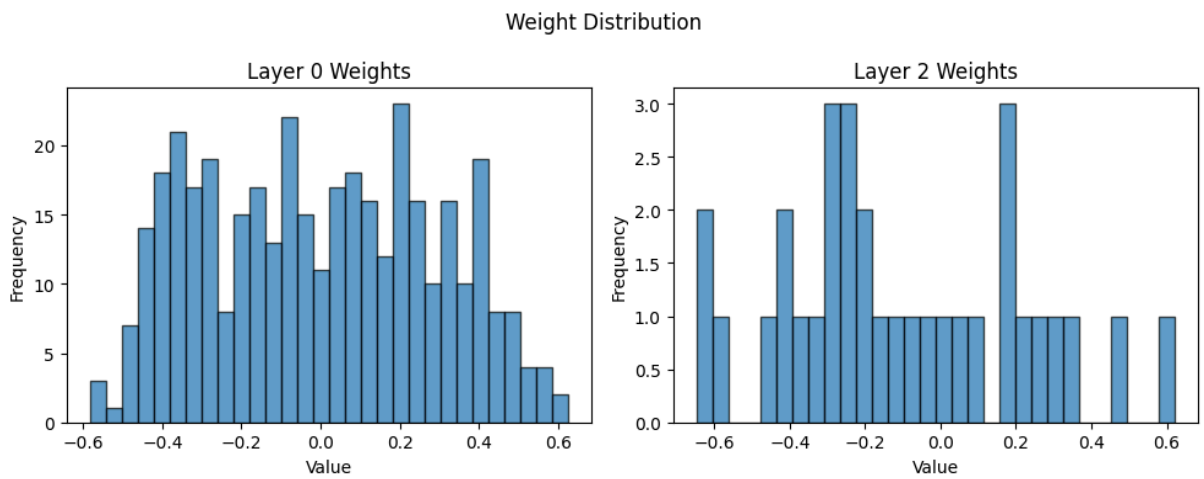

Weight Distribution
Layer 0 Weights
Layer 2 Weights
Frequency
Value

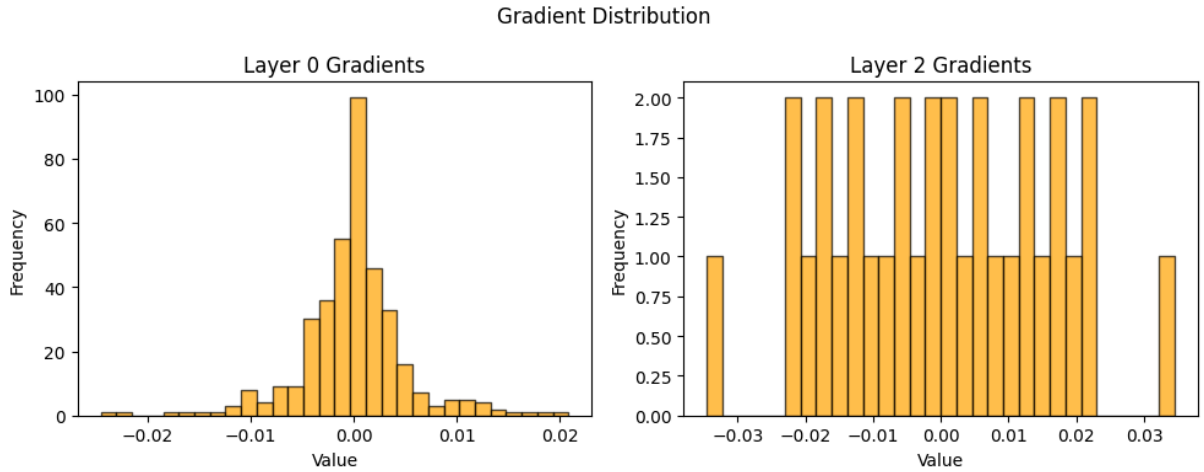

Gradient Distribution
Layer 0 Gradients
Layer 2 Gradients
Frequency
Value

v. Xavier Uniform



vi. Kaiming Uniform (He)



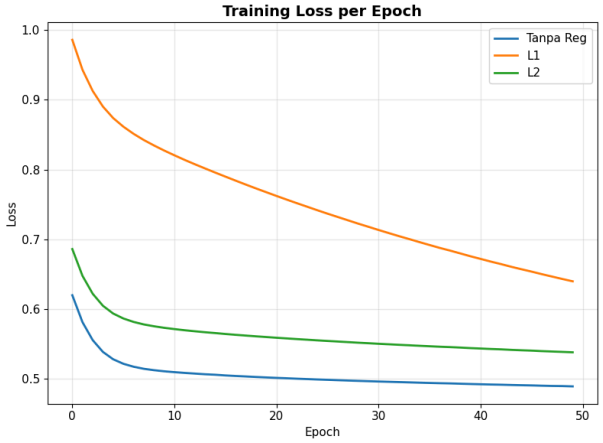


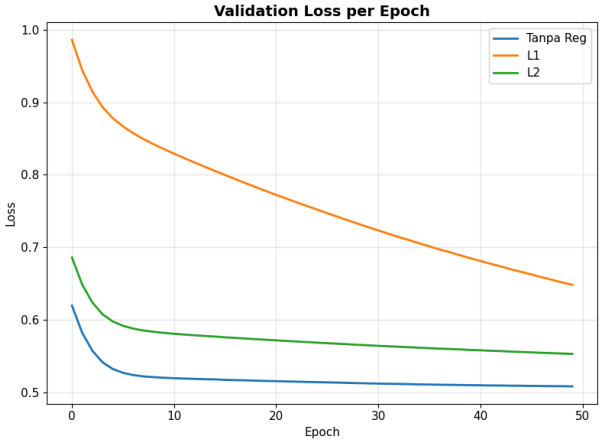
2.2.5 Pengaruh regularisasi

i. Perbandingan F1 Score

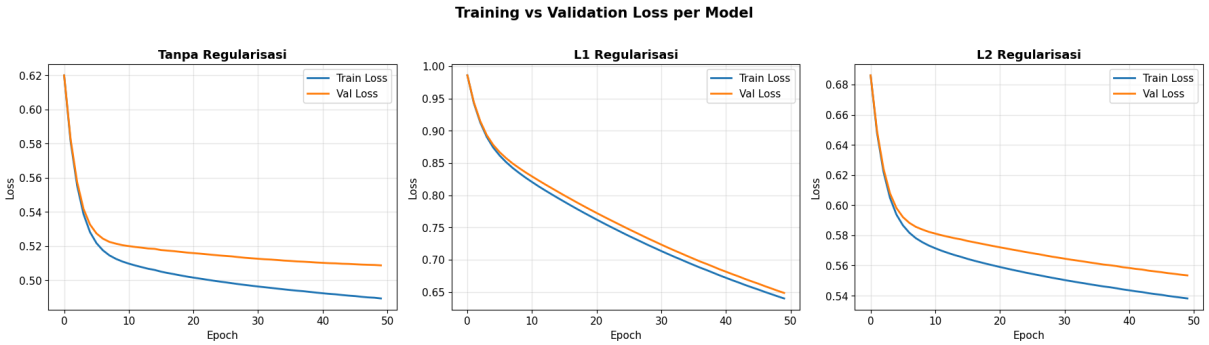
Model Variation	F1 Score
Tanpa Regularisasi	0.8134
L1 Regularisasi	0.8142
L2 Regularisasi	0.8156

ii. Training and Validation Loss per Epoch

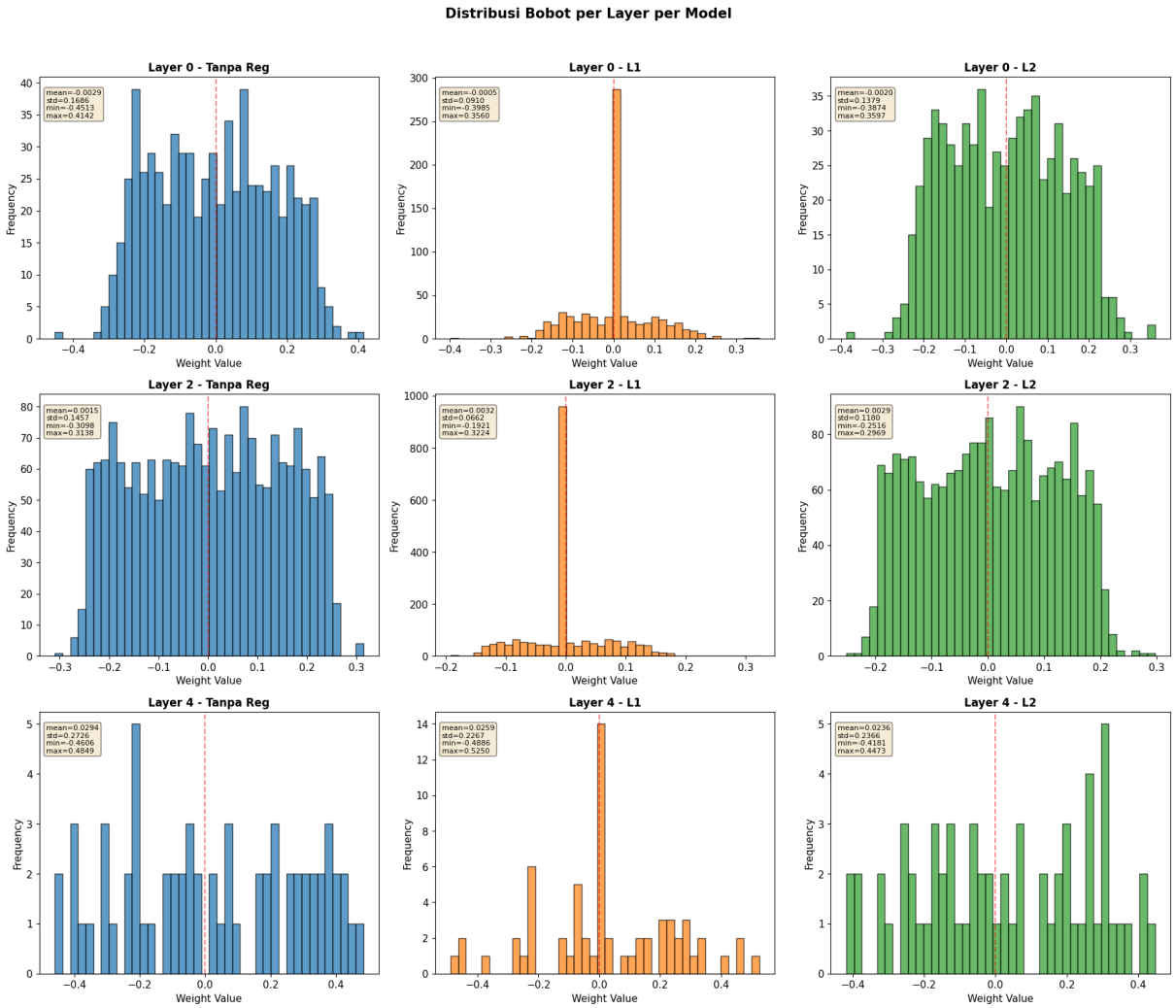




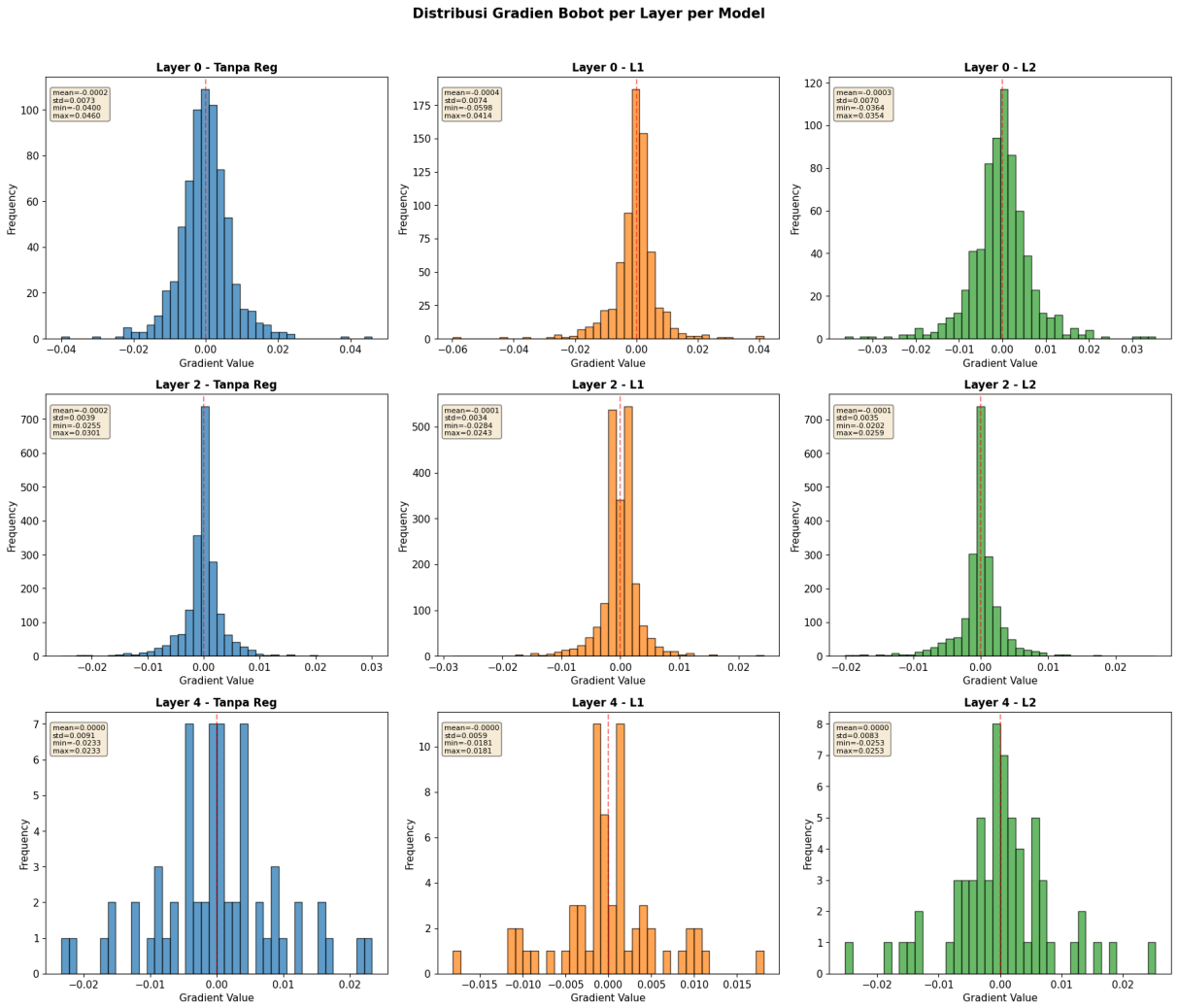
iii. Training and Validation Loss per Model



iv. Distribusi Bobot per Layer per Model



v. Distribusi Graadien Bobot per Layer per Model

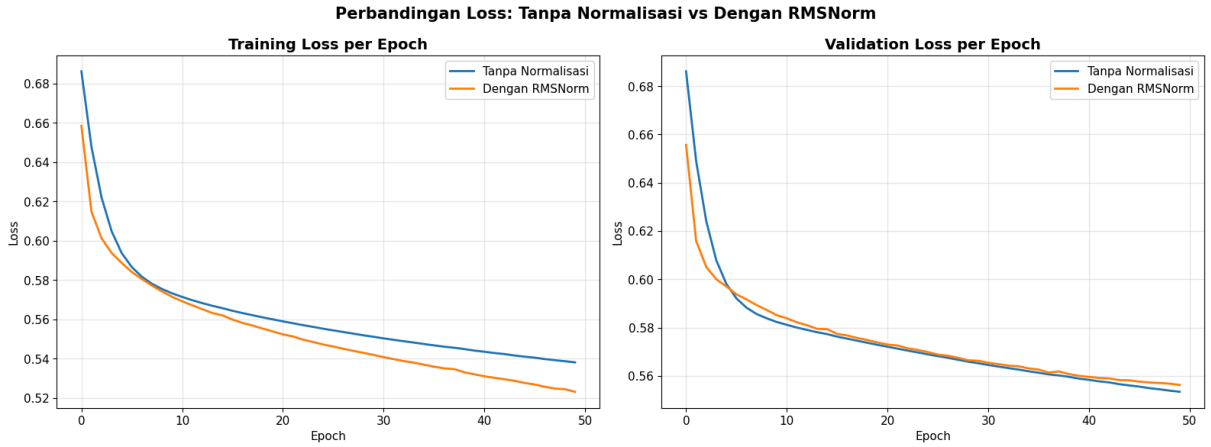


2.2.6 Pengaruh normalisasi RMSNorm

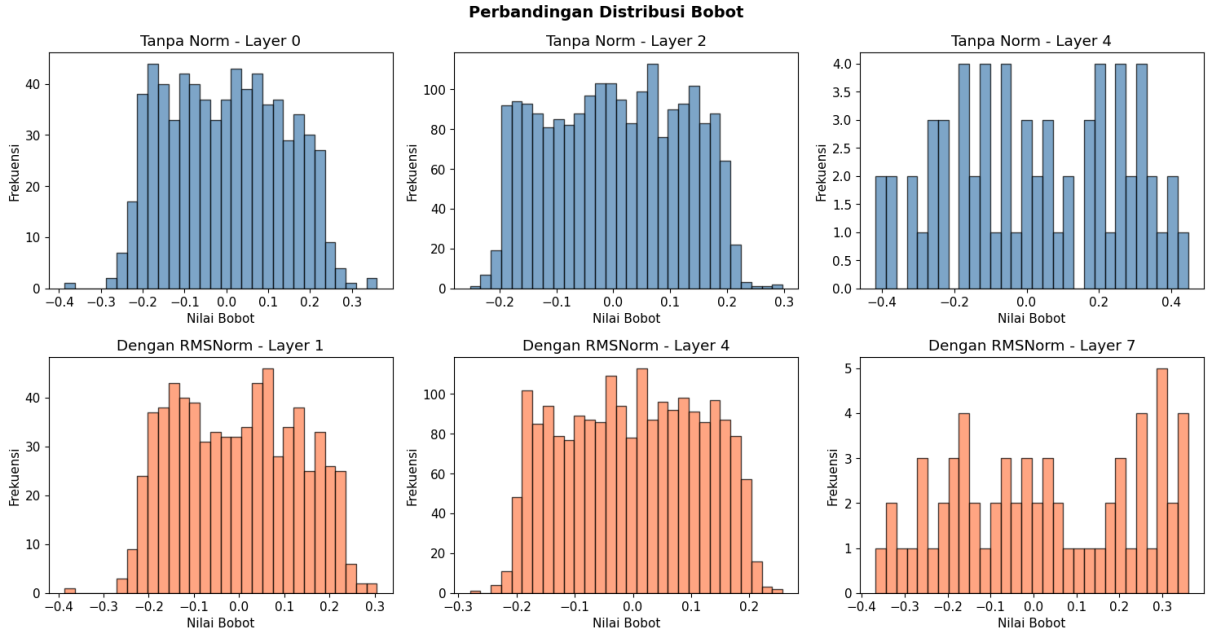
i. Perbandingan F1 Score

Model Variation	F1 Score
Tanpa Normalisasi	0.8156
RMSNorm	0.8104

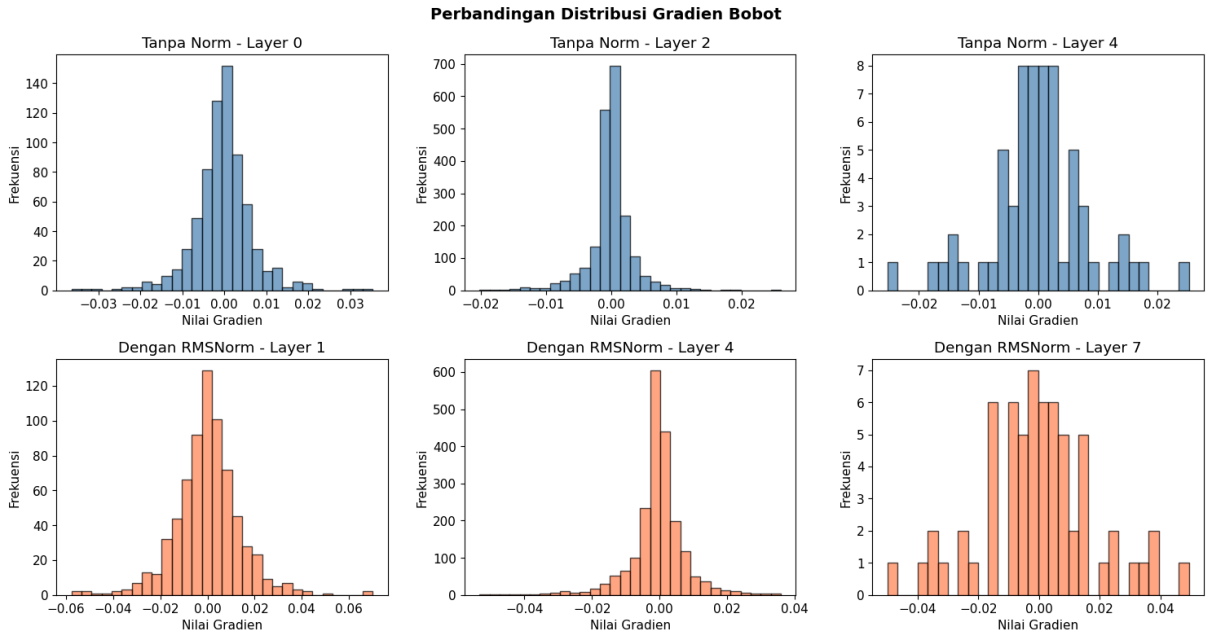
ii. Perbandingan Loss: Tanpa Normalisasi vs Dengan RMSNorm



iii. Perbandingan Distribusi Bobot



iv. Perbandingan Distribusi Gradien Bobot

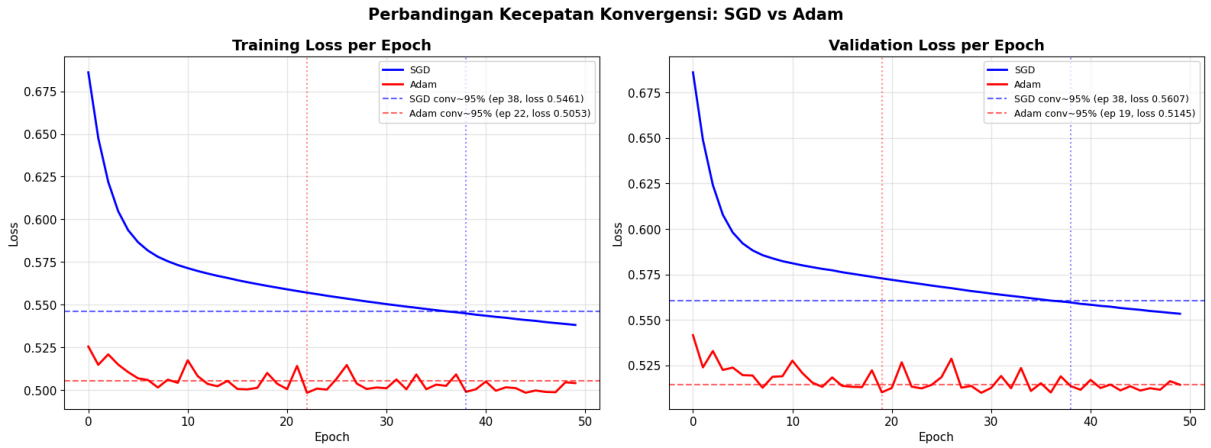


2.2.7 Perbandingan dengan Adam Optimizer

i. Perbandingan F1 Score

Model Variation	F1 Score
SGD	0.8156
Adam	0.8218

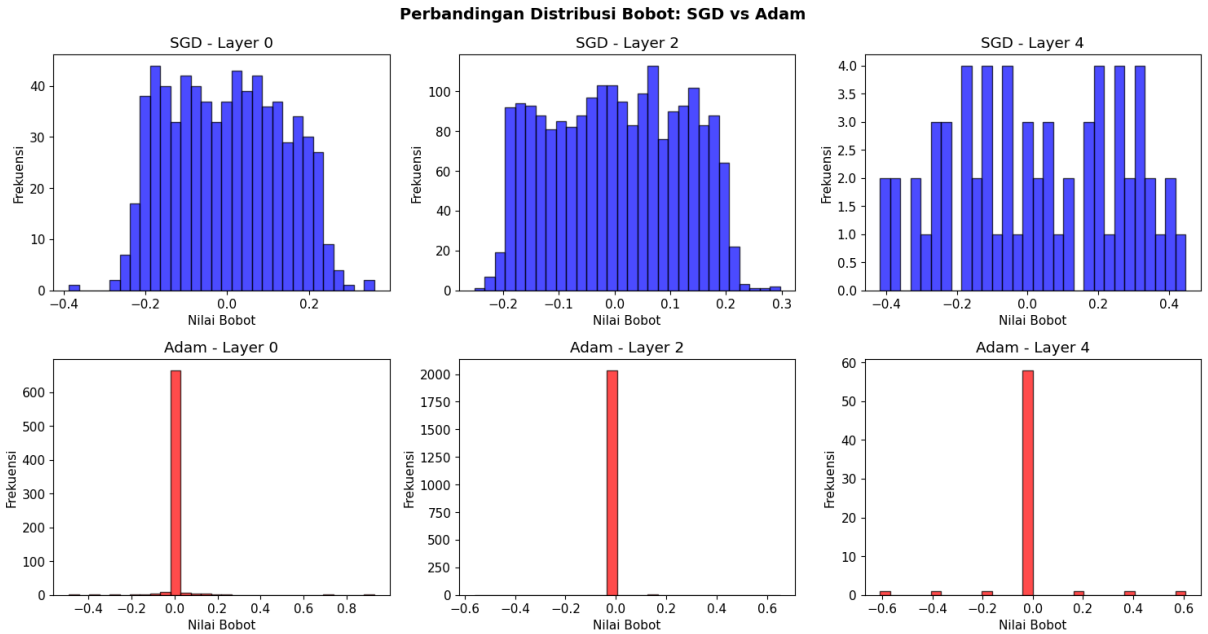
ii. Perbandingan Kecepatan Konvergensi: SGD vs Adam



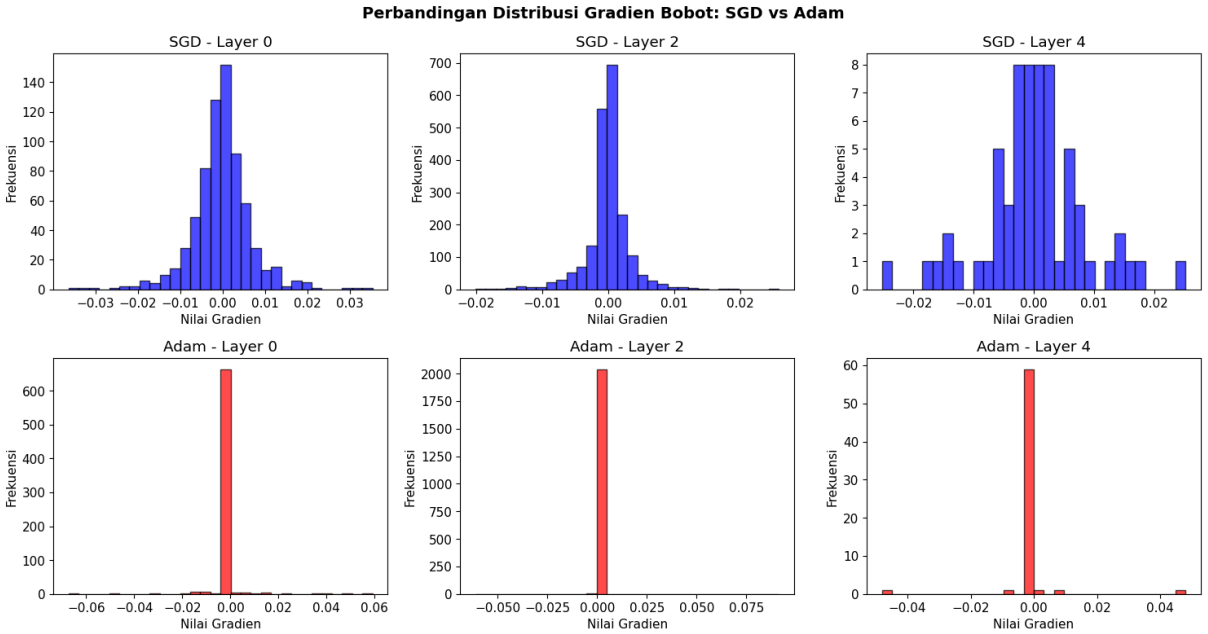
- Konvergen jika validation loss tidak membaik selama 10 epoch berturut-turut.
- Epoch konvergen dihitung dari validation loss, lalu dipakai sama untuk plot train dan val.

Model	Epoch Konvergen	Rule terpenuhi	Best Val Epoch	Best Val Loss	Epoch sejak best	Train loss @conv	Val loss @conv
SGD	49	False	49	0.553388	0	0.538108	0.553388
Adam	17	True	7	0.512737	10	0.501269	0.513061

iii. Perbandingan Distribusi Bobot: SGD vs Adam



iv. Perbandingan Distribusi Gradien Bobot: SGD vs Adam



2.2.8 Perbandingan dengan library sklearn

Perbandingan F1 Score

Model Variation	F1 Score
Model	0.7653

sklearn MLP	0.7620

3 Kesimpulan dan Saran

3.1 Kesimpulan

Berdasarkan implementasi dan serangkaian eksperimen yang telah dilakukan terhadap Feedforward Neural Network (FFNN) pada dataset Global Student Placement & Salary, dapat ditarik kesimpulan sebagai berikut.

Model FFNN yang dibangun dari awal menggunakan Python dan NumPy mampu menjalankan proses forward propagation, backward propagation, dan pembaruan bobot dengan benar. Hasil prediksi model kustom (F1 Score: 0.7653) sedikit melampaui implementasi MLPClassifier dari library scikit-learn (F1 Score: 0.7620) pada kondisi hyperparameter yang sama.

Depth dan width berpengaruh terhadap kapasitas belajar model, namun dengan diminishing returns. Menambah lebar (width) jaringan secara umum meningkatkan performa karena kapasitas representasi yang lebih besar, dengan width=64 memberikan F1 Score terbaik (0.7706) di antara variasi width. Sebaliknya, menambah kedalaman (depth) tidak meningkatkan performa; depth=8 menghasilkan F1 Score terburuk (0.6622). Terdapat kasus dimana kedalaman dapat meningkatkan performa model apabila tepat; depth=4 justru menghasilkan F1 Score terbaik (0.7579) di antara variasi depth lainnya. Pada depth=8, teridentifikasi gejala vanishing gradient yang ditunjukkan oleh nilai bobot dan gradien yang mendekati nol di layer-layer dalam, sehingga proses pembelajaran menjadi tidak stabil.

Pilihan fungsi aktivasi memengaruhi performa dan kestabilan pelatihan. ELU menghasilkan F1 Score tertinggi (0.7737), diikuti RMSNorm (0.7726) dan Leaky ReLU (0.7723). Fungsi aktivasi yang tidak "mematikan" neuron bernilai negatif, seperti ELU dan Leaky ReLU, terbukti lebih unggul dibandingkan ReLU standar (0.7636) karena menghindari masalah dying neuron. Sigmoid menghasilkan performa yang lebih rendah secara umum, konsisten dengan masalah vanishing gradient yang dimilikinya pada jaringan yang lebih dalam.

Learning rate memiliki pengaruh signifikan dan kritis terhadap konvergensi model. Learning rate sebesar 0.1 menghasilkan performa terbaik (F1 Score: 0.7953) dengan kurva loss yang turun mulus dan stabil. Learning rate terlalu tinggi (Lr=10, F1: 0.6690) menyebabkan model gagal konvergen dan menghasilkan prediksi yang buruk, sedangkan Lr=40 menyebabkan divergensi total yang terlihat dari lonjakan nilai loss yang ekstrem. Ini mengkonfirmasi bahwa pemilihan learning rate yang tepat merupakan salah satu faktor terpenting dalam pelatihan neural network.

Metode inisialisasi bobot secara nyata memengaruhi performa akhir model. Inisialisasi dengan distribusi normal memberikan F1 Score terbaik (0.8076), diikuti Kaiming Uniform (0.8016), Uniform (0.8029), dan Xavier Uniform (0.8005). Zero initialization menghasilkan performa terburuk (0.7620) karena seluruh neuron mendapat gradien yang identik (symmetry problem) sehingga jaringan tidak mampu belajar representasi yang beragam. Metode inisialisasi yang mempertimbangkan ukuran jaringan (Kaiming dan Xavier) memberikan awal yang lebih baik untuk proses optimisasi.

Regularisasi terbukti meningkatkan generalisasi model. L2 regularisasi menghasilkan F1 Score terbaik (0.8156), sedikit di atas L1 (0.8142) dan tanpa regularisasi (0.8134). Distribusi bobot menunjukkan bahwa L1 mendorong banyak bobot menuju nol (sparsity), sedangkan L2 mendistribusikan bobot secara lebih merata namun bernilai kecil (weight decay). Keduanya membantu mencegah overfitting, dengan L2 sedikit lebih unggul pada dataset ini.

RMSNorm mempercepat konvergensi di awal pelatihan, namun tidak meningkatkan F1 Score akhir. Model dengan RMSNorm (F1 Score: 0.8104) sedikit di bawah model tanpa normalisasi (0.8156). Meski demikian, grafik loss menunjukkan bahwa RMSNorm mempercepat penurunan training loss di epoch awal, yang dapat menjadi keunggulan pada skenario pelatihan dengan anggaran epoch terbatas. Distribusi bobot dengan RMSNorm cenderung lebih seragam di seluruh layer.

Adam optimizer menghasilkan performa lebih baik dan konvergensi lebih cepat dibanding SGD. Adam mencapai F1 Score 0.8218, melampaui SGD (0.8156). Grafik loss menunjukkan Adam konvergen jauh lebih cepat dan stabil, mencapai nilai loss rendah dalam 10 epoch pertama, sedangkan SGD masih terus menurun hingga epoch ke-50. Hal ini konsisten dengan keunggulan Adam dalam mengadaptasi learning rate secara individual untuk setiap parameter serta menggunakan bias correction untuk estimasi yang akurat di awal pelatihan.

3.2 Saran

Berdasarkan hasil eksperimen dan keterbatasan yang ditemukan, berikut adalah saran untuk pengembangan lebih lanjut.

1. Implementasikan Learning Rate Scheduler: Beberapa eksperimen menunjukkan training loss masih menurun di akhir epoch, sementara validation loss mulai naik. Penggunaan teknik seperti step decay, cosine annealing, atau reduce-on-plateau dapat membantu model konvergen ke titik yang lebih optimal dan mengurangi overfitting pada epoch lanjut.
2. Tambahkan mekanisme Early Stopping: Pada banyak eksperimen, terlihat bahwa validation loss mulai divergen dari training loss di epoch akhir (terutama epoch ke-20

pada berbagai variasi). Implementasi early stopping berdasarkan validation loss akan mencegah overfitting dan menghemat waktu komputasi tanpa mengorbankan performa generalisasi.

3. Eksplorasi teknik normalisasi lain seperti Batch Normalization atau Layer Normalization: RMSNorm yang diuji menunjukkan manfaat pada kecepatan konvergensi awal. Batch Normalization dan Layer Normalization adalah teknik yang terbukti efektif dalam melatih jaringan yang lebih dalam, dan perlu dibandingkan langsung dengan RMSNorm untuk menentukan teknik normalisasi yang paling sesuai untuk dataset dan arsitektur yang digunakan.
4. Lakukan feature engineering dan preprocessing yang lebih komprehensif: Performa model sangat bergantung pada kualitas data input. Eksplorasi lebih lanjut pada teknik seperti feature selection, penanganan imbalanced data, dan normalisasi fitur yang lebih teliti dapat meningkatkan F1 Score secara signifikan di luar optimisasi arsitektur model saja.

4 Pembagian tugas tiap anggota kelompok

NIM	Pembagian Tugas
13523006	Semua
13523008	Semua
13523032	Semua

5 Referensi

https://www.jasonosajima.com/forwardprop

https://www.jasonosajima.com/backprop

https://numpy.org/doc/2.2/

https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

https://math.libretexts.org/Bookshelves/Calculus/Calculus_%28OpenStax%29/14%3A_Differentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariable_Functions

https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/

https://www.geeksforgeeks.org/training-and-validation-loss-in-deep-learning/

https://builtin.com/data-science/l2-regularization

https://arxiv.org/pdf/1412.6980

6 Lampiran Form Penggunaan AI

1. Varel Tiara

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:
Nama/NIM : Varel Tiara/13523008
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika / STEI
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Tugas Besar 1 IF3270 Pembelajaran Mesin Feedforward Neural Network

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	1
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Tidak	1
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	1
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 15 Maret 2026

Varel Tiara

13523008

2. Nathan Jovial Hartono

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM : Nathan Jovial Hartono/13523032
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika / STEI
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Tugas Besar 1 IF3270 Pembelajaran Mesin Feedforward Neural Network

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	1
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Tidak	1
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	1
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 16 Maret 2026

Nathan Jovial Hartono
13523032

3. William Andrian Dharma T

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM : William Andrian Dharma T/13523006
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika / STEI
Kode Mata Kuliah : IF3270
Nama Mata Kuliah : Pembelajaran Mesin
Judul Tugas/Proposal : Tugas Besar 1 IF3270 Pembelajaran Mesin Feedforward Neural Network

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	1
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	1
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Tidak	1
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	1
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 16 Maret 2026

William Andrian Dharma T

13523006